1 一、PositionEncoding

- **论文地址**

> **https://arxiv.org/pdf/1706.03762**

1.1 位置编码介绍

- 位置编码在Transformer模型中是用来将位置信息与输入序列相结合的一种技术。在传统的序列模型（如RNN）中，输入的序列数据是有顺序的，模型本身能够感知输入数据的顺序。然而，Transformer是一种不具备内置顺序敏感性的模型，因此需要通过位置编码来将位置信息显式地添加到输入数据中。

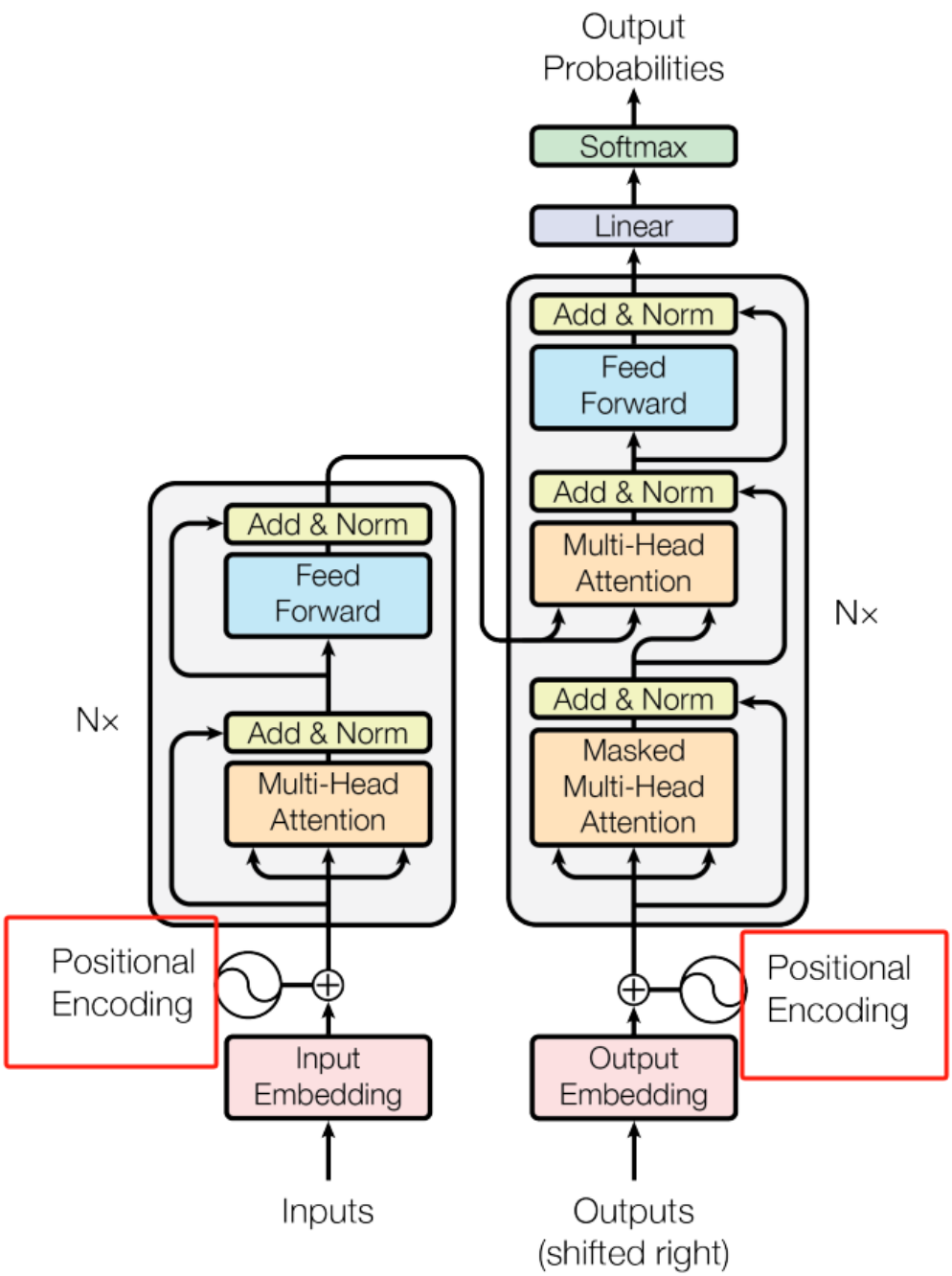


Figure 1: The Transformer - model architecture.

1.2 位置编码的数学公式

- 在Transformer中，位置编码通过一组确定性的正弦和余弦函数的组合来产生。这种编码方式允许模型学习位置信息，同时也保留了相对位置的不变性。具体的数学公式如下

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

$$PE_(pos, 2i) = \sin(\frac{pos}{10000^{\frac{2i}{d_{model}}}})$$
$$PE_(pos, 2i+1) = \cos(\frac{pos}{10000^{\frac{2i}{d_{model}}}}) \tag{1}$$

其中，pos 表示位置，i 是维度索引，d_{model} 是模型的维度 (即embedding的维度)。

1.3 代码

- 根据上述公式，实现 `PositionlEncoding`

```python
from torch import nn
import torch

class PositionEncoding(nn.Module):
    """位置编码"""

    def __init__(self, d_model, max_seq_len=512):
        super(PositionEncoding, self).__init__()

        # shape[max_seq_len]
        position = torch.arange(0, max_seq_len)
        # shape[max_seq_len, 1]
        position = position.unsqueeze(1)  # 升维

        # shape [d_model/2]
        item = 1 / 10000 ** (torch.arange(0, d_model, 2) / d_model)

        # shape[max_seq_len, d_model/2]
        tmp_pos = position * item

        # shape[max_seq_len, d_model]
        pe = torch.zeros(max_seq_len, d_model)

        pe[:, 0::2] = torch.sin(tmp_pos)  # 偶数
        pe[:, 1::2] = torch.cos(tmp_pos)  # 奇数

        # shape: [max_seq_len, d_model] --> [batch, max_seq_len, d_model]
        pe = pe.unsqueeze(0)  # 在第一个维度上升维

        # 将位置编码pe注册为缓冲区
        self.register_buffer(name="pe", tensor=pe, persistent=False)

    def forward(self, x):
        batch, seq_len, d_model = x.shape
```

```
        pe = self.pe
        return x + pe[:, :seq_len, :]
```

- 相关解释

    1. `初始化参数解释`：

       d_model : 模型维度（embedding维度）

       max_seq_len : 是模型支持的最大序列长度，确保模型能够处理任意长度不超过 max_seq_len 的序列。

    2. `注册缓冲区`

       `self.register_buffer` 方法

       通过将位置编码 pe 注册为缓冲区，可以在模型的前向传播中直接使用 self.pe，而不需要每次都重新计算。
       缓冲区不会被优化器更新，因此位置编码在训练过程中保持不变，这符合位置编码的设计初衷。

    3. `前向传播参数`

       x : 输入序列，shape为 [batch_size, seq_len, d_model]

1.4 使用示例

- 测试代码

```
# 假设我们有一个大小为 (batch_size=32, seq_len=20, d_model=512) 的输入
batch_size = 32
seq_len = 20
d_model = 512

# 创建位置编码的实例
pos_encoding_layer = PositionEncoding(d_model=d_model)

# 生成随机输入数据
input_data = torch.randn(batch_size, seq_len, d_model)

# 获取带有位置编码的数据
output_data = pos_encoding_layer(input_data)

print(output_data.shape)  # 输出应为 (32, 20, 512)
```

创建一个 `PositionEncoding` 类的实例，并通过传入一个随机生成的输入张量来获取添加了位置信息的输出张量。

注意，PositionalEncoding 不改变输入序列的长度

2 一、PositionalEncoding (对数空间计算实现)

- 参考论文： `https://arxiv.org/pdf/1706.03762`

> https://arxiv.org/pdf/1706.03762

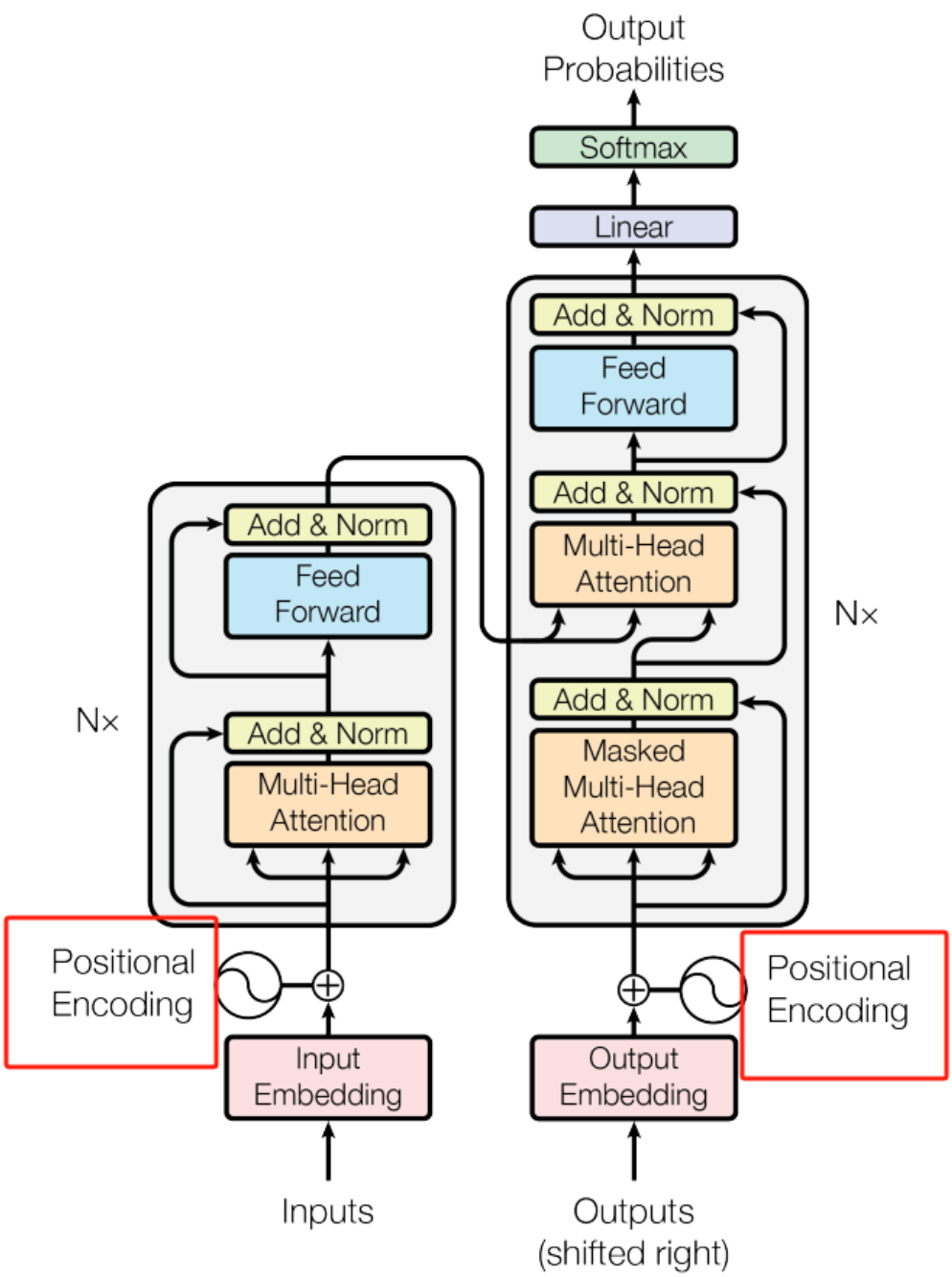


Figure 1: The Transformer - model architecture.

位置编码的正常实现链接如下

> https://blog.csdn.net/hbkybkzw/article/details/147431820

这次说的是位置编码的另一种实现方式，在计算分母的幂次时使用对数变换技巧，在数值上更稳定，两者只是在计算上不同，实际是等价的

2.1 数学公式

- 原公式

$$
\begin{aligned}
PE_{(pos,2i)} &= \sin(\frac{pos}{10000^{\frac{2i}{d_{model}}}}) \\
PE_{(pos,2i+1)} &= \cos(\frac{pos}{10000^{\frac{2i}{d_{model}}}})
\end{aligned}
\tag{2}
$$

其中，pos 表示位置，i 是维度索引，d_{model} 是模型的维度 (即embedding的维度)。

- 分母的变换

已知

$$
x = e^{\log_{(} x)} \tag{3}
$$

则

$$
\begin{aligned}
\frac{1}{10000^{\frac{2i}{d_{model}}}} &= 10000^{-\frac{2i}{d_model}} \\
&= e^{\log 10000^{-\frac{2i}{d_model}}} \\
&= e^{-\frac{2i}{d_model} \log 10000} \\
&= e^{-2i \log 10000 / d_{model}}
\end{aligned}
\tag{4}
$$

- 所以原来的实现代码为

```
item = 1 / 10000 ** (torch.arange(0, d_model, 2) / d_model)
```

更改为对数计算后是

```
div_item = torch.exp(
            torch.arange(0, d_model, 2) * -(math.log(10000.0)) / d_model
        )
```

2.2 代码实现 (对数空间计算)

- 根据上述公式，`PositionalEncoding` 的一种实现方式如下，其中 `div_term` 的计算在对数空间进行，以提高数值稳定性：

```
import torch
from torch import nn
import math

class PositionalEncodingExponent(nn.Module):
    "Implement the PE function."

    def __init__(self, d_model, max_seq_len=512):
```

```python
        super(PositionalEncodingExponent, self).__init__()
        # shape of position : [max_seq_len, 1]
        position = torch.arange(0, max_seq_len).unsqueeze(1)


        # Compute the positional encodings once in log space.
        div_item = torch.exp(
            torch.arange(0, d_model, 2) * -(math.log(10000.0)) / d_model
        )
        tmp_pos = position * div_item

        pe = torch.zeros(max_seq_len, d_model)

        pe[:, 0::2] = torch.sin(tmp_pos)
        pe[:, 1::2] = torch.cos(tmp_pos)
        pe = pe.unsqueeze(0)
        self.register_buffer("pe", pe,persistent=False)

    def forward(self, x):
        batch,seq_len,d_model = x.shape # 注意，这里的x已经经过embedding
        pe = self.pe
        return x + pe[:, :seq_len, :]

```

2.3 使用示例

- 测试代码

```python
# 假设我们有一个大小为 (batch_size=32, seq_len=20, d_model=512) 的输入
batch_size = 32
seq_len = 20
d_model = 512

# 创建位置编码的实例
pos_encoding_layer = PositionalEncodingExponent(d_model=d_model)

# 生成随机输入数据
input_data = torch.randn(batch_size, seq_len, d_model)

# 获取带有位置编码的数据
output_data = pos_encoding_layer(input_data)

print(output_data.shape)  # 输出应为 (32, 20, 512)
```

此示例创建了一个 `PositionalEncoding` 类的实例，并将其应用于一个随机生成的输入张量，模拟了在Transformer模型中将位置信息添加到词嵌入的过程。输出张量的形状与输入张量保持一致。

3 一、Rotary Position Embedding (RoPE)

- **相关论文地址**

> https://arxiv.org/abs/2104.09864

3.1 RoPE介绍

- 扭转位置编码（Rotary Position Embedding, RoPE）是一种改进的位置编码方法，用于增强模型对序列顺序的敏感性。它通过对输入序列的特征向量进行旋转变换，将位置信息隐式编码进来，使得模型能够更好地捕获相对位置信息。RoPE 通常用于对比 Transformer 中传统的位置编码，能够提供更强的位置感知能力。

3.2 RoPE的数学公式

- RoPE 使用了旋转矩阵来对输入向量进行变换。具体公式如下：

$$\mathrm{RoPE}(x) = \begin{bmatrix} \cos(\theta_{pos}) & -\sin(\theta_{pos}) \\ \sin(\theta_{pos}) & +\cos(\theta_{pos}) \end{bmatrix} \begin{bmatrix} \mathbf{x}_{2i} \\ \mathbf{x}_{2i+1} \end{bmatrix} \tag{5}$$

其中，θ_{pos} 是与位置 pos 相关的旋转角度，定义为：

$$\theta_{pos} = pos \cdot \frac{1}{10000^{2i/d_{model}}} \tag{6}$$

在此公式中，pos 表示位置索引，d_{model} 是模型的维度。

3.3 代码实现

- 基于上述公式，实现 `RoPE` 。

```python
import torch
from torch import nn

class RotaryPositionEmbedding(nn.Module):
    """Rotary Position Embedding (RoPE)"""

    def __init__(self, d_model):
        super().__init__()
        self.d_model = d_model

        ## shape: [d_model //2 ]
        self.theta = 1 / 10000 ** (torch.arange(0, d_model, 2) / d_model)

    def forward(self, x):
        """
        shape of x: [batch_size, seq_len, d_model]
        """
        batch_size, seq_len, d_model = x.shape

```

```python
        # 创建位置编码矩阵 [seq_len, 1]
        pos = torch.arange(seq_len).unsqueeze(1)

        # 计算旋转角度 [seq_len, d_model//2]
        angle = pos * self.theta
        cos_angle = torch.cos(angle)
        sin_angle = torch.sin(angle)

        # 分割奇偶维度
        x_even = x[..., 0::2]  # 偶数维度 [batch_size, seq_len, d_model//2]
        x_odd = x[..., 1::2]  # 奇数维度 [batch_size, seq_len, d_model//2]

        # 应用旋转矩阵
        x_rotated = torch.zeros_like(x)
        x_rotated[..., 0::2] = x_even * cos_angle - x_odd * sin_angle
        x_rotated[..., 1::2] = x_even * sin_angle + x_odd * cos_angle
        return x_rotated
```

- 相关解释
  1. `初始化参数解释`：
     - **d_model**: 模型维度。
  2. `forward方法说明`：
     - 计算每个位置 pos 的旋转角度 θ_{pos}。
     - 对输入 x 的偶数和奇数索引分别进行旋转变换。
     - 返回经过RoPE处理后的向量。
- 相关维度变化

操作步骤	张量形状变化示例
输入数据	[batch, seq_len, d_model]
创建位置编码矩阵	[seq_len, 1]
计算旋转角度	[seq_len, d_model//2]
分割奇偶维度	2x [batch, seq_len, d_model//2]
应用旋转矩阵	2x [batch, seq_len, d_model//2]
合并旋转结果	[batch, seq_len, d_model]

3.4 使用示例

- 测试代码

```python
if __name__ == "__main__":

    batch_size = 4
    seq_len = 100
    d_model = 512

    rope_layer = RotaryPositionEmbedding(d_model=d_model)

    input_data = torch.randn(batch_size, seq_len, d_model)

    output_data = rope_layer(input_data)
    print(output_data.shape)  # 输出应为 (4, 100, 512)
```

在这里，我们创建了一个 `RotaryPositionEmbedding` 类的实例，并通过传入一个随机生成的输入张量来获取经过RoPE处理的输出张量。RoPE 同样不会改变输入序列的长度。

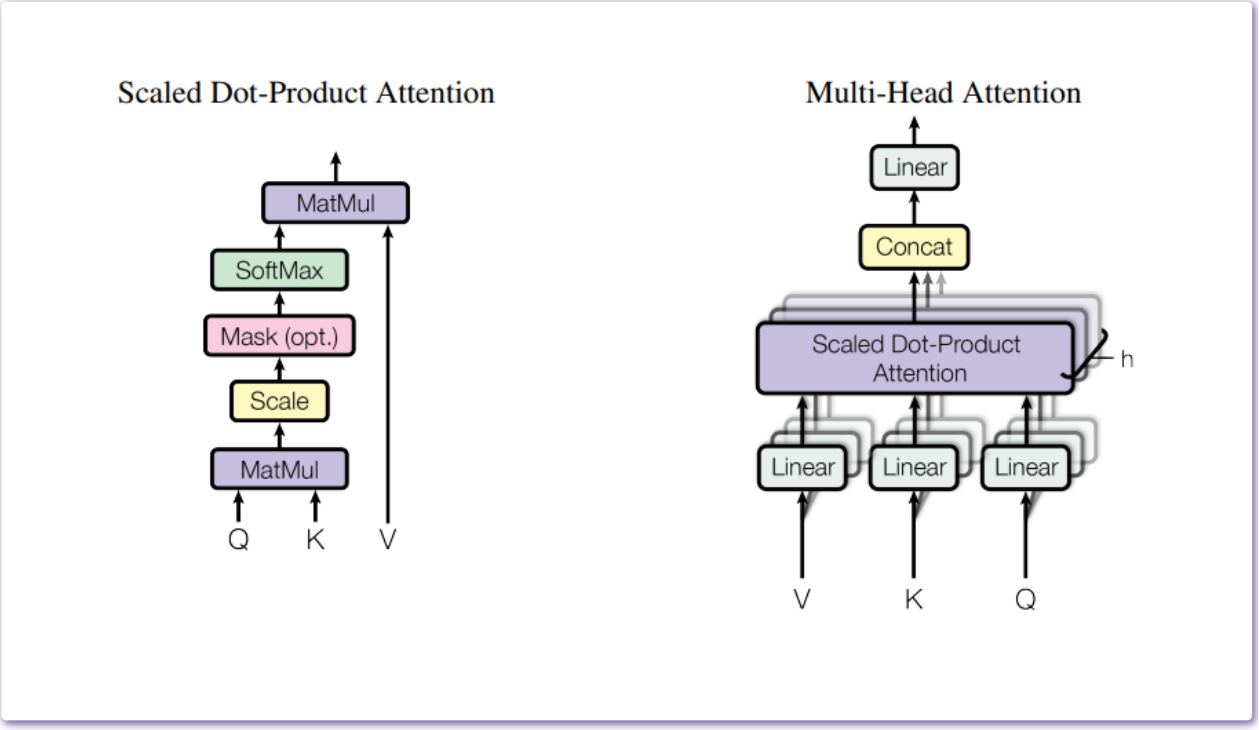
4 二、Scaled Dot-Product Attention

- **论文地址**

> https://arxiv.org/pdf/1706.03762

4.1 注意力机制介绍

- 缩放点积注意力是Transformer模型的核心组件，用于计算序列中不同位置之间的关联程度。其核心思想是通过查询向量（query）和键向量（key）的点积来获取注意力分数，再通过缩放和归一化处理，最后与值向量（value）加权求和得到最终表示。



4.2 数学公式

- 缩放点积注意力的计算过程可分为三个关键步骤：
    1. **点积计算与缩放**：通过矩阵乘法计算查询向量与键向量的相似度，并使用 $\sqrt{d_k}$ 缩放
    2. **掩码处理**（可选）：对需要忽略的位置施加极大负值掩码
    3. **Softmax归一化**：将注意力分数转换为概率分布
    4. **加权求和**：用注意力权重对值向量进行加权

  公式表达为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{7}$$

  其中：

  - $Q \in \mathbb{R}^{seq_len \times d_model}$：查询矩阵

- $K \in \mathbb{R}^{seq_len \times d_model}$：键矩阵
- $V \in \mathbb{R}^{seq_len \times d_model}$：值矩阵

seq_len 为序列长度，d_model 为embedding的维度。

4.3 代码实现

- 计算注意力分数

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import torch

def calculate_attention(query, key, value, mask=None):
    """计算缩放点积注意力分数

    参数说明:
    query: [batch_size, n_heads, seq_len, d_k]
    key:   [batch_size, n_heads, seq_len, d_k]
    value: [batch_size, n_heads, seq_len, d_k]
    mask:  [batch_size, seq_len, seq_len] (可选)
    """
    d_k = key.shape[-1]
    key_transpose = key.transpose(-2, -1)  # 转置最后两个维度

    # 计算缩放点积 [batch, h, seq_len, seq_len]
    att_scaled = torch.matmul(query, key_transpose) / d_k ** 0.5

    # 掩码处理（解码器自注意力使用）
    if mask is not None:
        att_scaled = att_scaled.masked_fill_(mask=mask, value=-1e9)

    # Softmax归一化
    att_softmax = torch.softmax(att_scaled, dim=-1)

    # 加权求和 [batch, h, seq_len, d_k]
    return torch.matmul(att_softmax, value)
```

- 相关解释

1. **输入张量 query, key, value的形状**

   如果是直接计算的话，那么shape是 [batch_size, seq_len, d_model]

   当然为了学习更多的表征，一般都是多头注意力，这时候shape则是[batch_size, n_heads, seq_len, d_k]

   其中

   - batch_size：批量
   - n_heads：注意力头的数量
   - seq_len: 序列的长度
   - d_model: embedding维度
   - d_k： d_k = d_model / n_heads

2. 代码中的shape转变

- **key_transpose** :key的转置矩阵

  由 key 转置了最后两个维度，维度从 [batch_size, n_heads, seq_len, d_k] 转变为 [batch_size, n_heads, d_k, seq_len]

- **att_scaled** ： 缩放点积

  由 query 和 key 通过矩阵相乘得到

  [batch_size, n_heads, seq_len, d_k] @ [batch_size, n_heads, d_k, seq_len] --> [batch_size, n_heads, seq_len, seq_len]

- **att_score**： 注意力分数

  由两个矩阵相乘得到

  [batch_size, n_heads, seq_len, seq_len] @ [batch_size, n_heads, seq_len, d_k] --> [batch_size, n_heads, seq_len, d_k]

4.4 使用示例

- 测试代码

```python
if __name__ == "__main__":
    # 模拟输入：batch_size=2, 8个注意力头，序列长度512，d_k=64
    x = torch.ones((2, 8, 512, 64))

    # 计算注意力（未使用掩码）
    att_score = calculate_attention(x, x, x)

    print("输出形状:", att_score.shape)  # torch.Size([2, 8, 512, 64])
    print("注意力分数示例:\n", att_score[0,0,:3,:3])
```

在实际使用中通常会将此实现封装为 `nn.Module` 并与位置编码、残差连接等组件配合使用，构建完整的Transformer层。

---

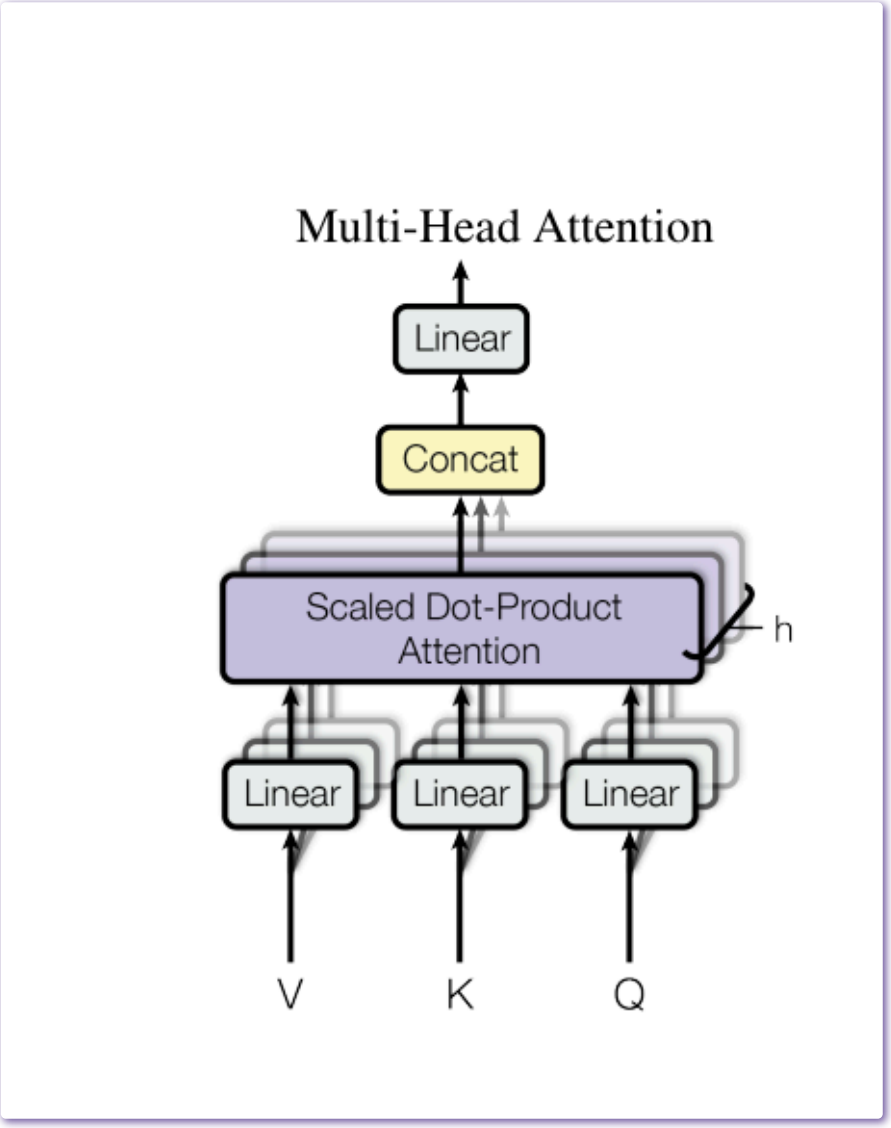
5 三、Multi-Head Attention

- **论文地址**

> https://arxiv.org/pdf/1706.03762

5.1 多头注意力介绍

- 多头注意力是Transformer模型的关键创新，通过并行执行多个独立的注意力计算单元，使模型能够同时关注来自不同表示子空间的信息。每个注意力头学习不同的语义特征，最后通过线性变换将多头的输出组合为最终结果。



当n_heads=1时，多头注意力等价于标准缩放点积注意力。多头设计通过建立多个独立的"观察视角"，使模型能够捕获更丰富的上下文信息。

5.2 数学公式

- 多头注意力通过以下三个步骤实现：
    1. **线性投影**：对Q/K/V进行线性变换

2. **并行注意力计算**：在多个子空间中计算缩放点积注意力
3. **输出融合**：拼接多头结果并进行线性变换

公式表达为：

$$\begin{aligned}\text{MultiHead}(Q,K,V) &= \text{Concat}(head_1,\ldots,head_h)W^O \\ \text{where } head_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned} \tag{8}$$

其中：

- $W_i^Q \in \mathbb{R}^{seq_len \times d_k}$：第i个头的查询投影矩阵
- $W_i^K \in \mathbb{R}^{seq_len \times d_k}$：第i个头的键投影矩阵
- $W_i^V \in \mathbb{R}^{seq_len \times d_k}$：第i个头的值投影矩阵
- $W^O \in \mathbb{R}^{seq_len \times d_model}$：输出投影矩阵

seq_len 为序列长度，d_model 为embedding的维度, d_k 为每个注意力头的维度，假设头数量为n_heads ,且n_heads能被d_model给整数，则

$$d_k = \frac{d_model}{n_heads} \tag{9}$$

5.3 代码实现

- 需要导入之前写的计算注意力分数的函数

> https://blog.csdn.net/hbkybkzw/article/details/147462845

```python
import torch
from torch import nn

def calculate_attention(query, key, value, mask=None):
    d_k = key.shape[-1]
    att_scaled = torch.matmul(query, key.transpose(-2, -1)) / d_k ** 0.5  # 缩放点积,注意区分这里的 d_k 是多头注意力中的“d_model”

    if mask is not None:
        att_scaled = att_scaled.masked_fill_(mask=mask, value=-1e9)  # 或者value= -inf,这样在softmax的时候，掩码的部分就为0了

    att_softmax = torch.softmax(input=att_scaled, dim=-1)  # softmax
    att_score = torch.matmul(att_softmax, value)  # 注意力分数
    return att_score
```

- 多头注意力模块

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, n_heads, d_model, dropout_prob=0.1):
        super(MultiHeadAttention, self).__init__()
        assert d_model % n_heads == 0  # 需要确保d_model一定要能被注意力头的数量整除

        # 定义四个全连接层（Q/K/V/Output）
```

```python
        self.q_linear = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
        self.k_linear = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
        self.v_linear = nn.Linear(in_features=d_model, out_features=d_model, bias=False)
        self.linear = nn.Linear(in_features=d_model, out_features=d_model, bias=False)

        self.dropout = nn.Dropout(dropout_prob)

        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        self.d_model = d_model

    def forward(self, q, k, v, mask=None):
        """前向传播过程

        输入形状：[batch_size, seq_len, d_model]
        输出形状：[batch_size, seq_len, d_model]
        """
        # 步骤1：线性投影并分割多头
        q = self.q_linear(q)
        q = q.reshape(q.shape[0], -1, self.n_heads, self.d_k)  # [batch_size,seq_len,n_heads,d_k]
        q = q.transpose(1, 2)  # [batch_size, n_heads, seq_len, d_k]
        # 等价于
        # q = self.q_linear(q).reshape(q.shape[0],-1,self.n_heads,self.d_k).transpose(1,2)

        k = self.k_linear(k)
        k = k.reshape(k.shape[0], -1, self.n_heads, self.d_k)
        k = k.transpose(1, 2)

        v = self.v_linear(v)
        v = v.reshape(v.shape[0], -1, self.n_heads, self.d_k)
        v = v.transpose(1, 2)


        # 步骤2：计算注意力（每个头独立计算）
        out = calculate_attention(q, k, v, mask)  # [batch_size,seq_len,n_heads,d_k]

        # 步骤3：拼接多头结果并进行输出投影
        out = out.transpose(1, 2)  # [batch_size,n_heads,seq_len,d_k]
        out = out.reshape(out.shape[0], -1, self.d_model)  # [batch_size,seq_len,d_model]
        out = self.linear(out)
        out = self.dropout(out)
        return out

```

- 关键操作解析

  维度变换流程

操作步骤	张量形状变化示例		
输入数据	[batch_size, seq_len, d_model]		
步骤1-线性投影（保持维度）	[batch_size, seq_len, d_model]		
步骤1-分割多头（n_heads）	[batch_size, seq_len, n_heads, d_k]		
步骤1-维度转置（交换头与序列轴）	[batch_size, n_heads , seq_len, d_k]		
步骤2-计算注意力（各头独立）	[batch_size, n_heads , seq_len, d_k]		
步骤3-维度转置（交换头与序列轴）	[batch_size , seq_len, n_heads ,d_k]		
步骤3-拼接多头（恢复原始维度）	[batch_size, seq_len, d_model]		

5.4 使用示例

- 测试代码

```python
if __name__ == "__main__":
    # 实例化模块：8头注意力，512维模型，20% dropout
    multihead_attention = MultiHeadAttention(n_heads=8, d_model=512, dropout_prob=0.2)

    # 模拟输入：batch_size=4，序列长度100，维度512
    x = torch.randn(4, 100, 512)

    # 前向传播（自注意力模式）
    output = multihead_attention(x, x, x)

    print("输入形状:", x.shape)        # torch.Size([4, 100, 512])
    print("输出形状:", output.shape)   # torch.Size([4, 100, 512])
    print("输出范数:", torch.norm(output))  # 约1.2-1.8（取决于初始化）
```

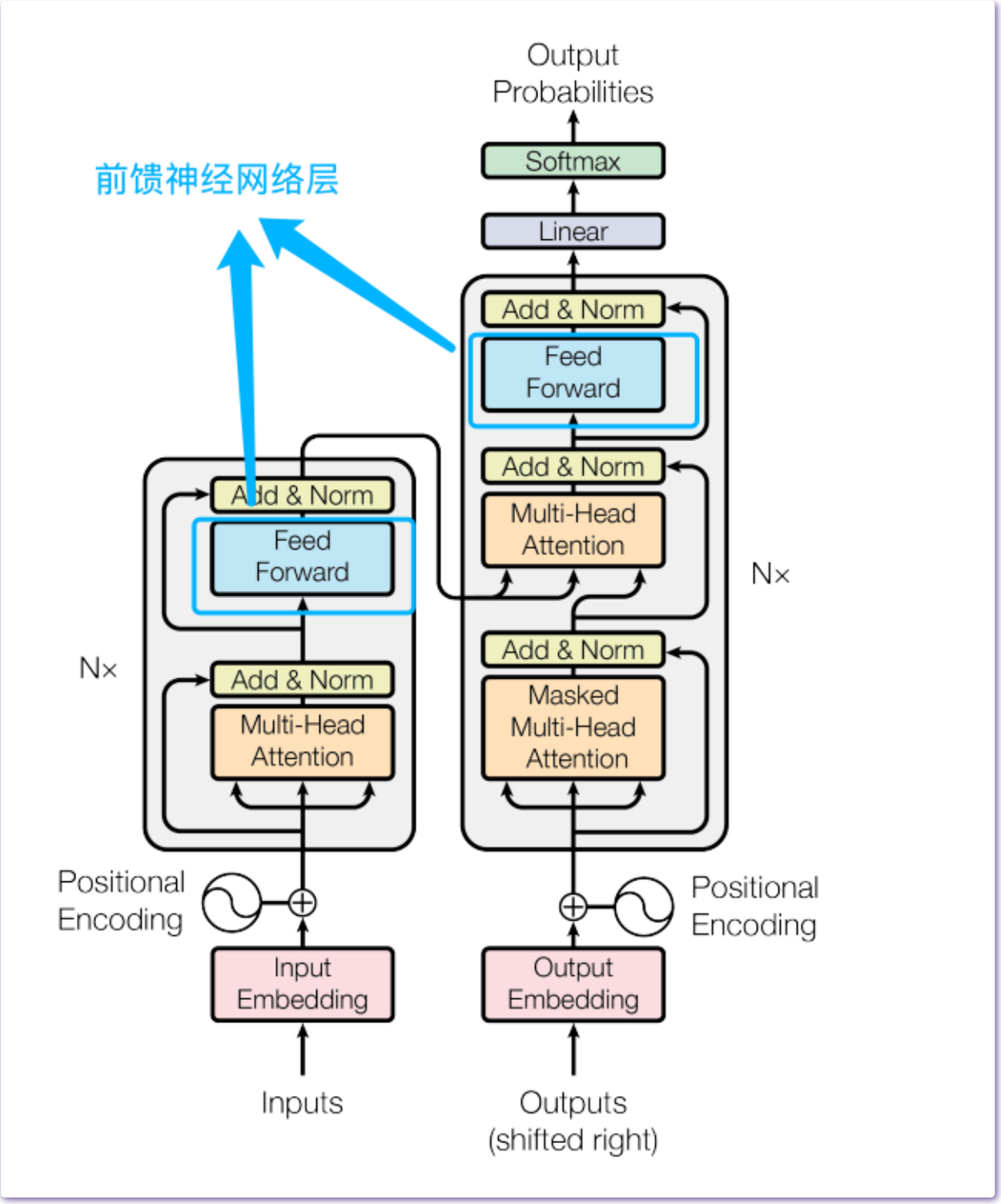
6 四、FeedForward Network

- **论文地址**

> **https://arxiv.org/pdf/1706.03762**

6.1 前馈网络介绍

- 前馈网络是Transformer模型中的关键组件，每个Transformer层包含一个多头注意力模块和一个前馈网络模块。该模块通过两次线性变换和激活函数，为模型提供非线性建模能力。其核心设计是通过中间隐藏层的维度扩展（ffn_hidden > d_model），增强模型的特征表达能力。



典型配置中ffn_hidden=2048（当d_model=512时），这种"扩展-收缩"结构帮助模型捕获更复杂的模式。前馈网络独立作用于每个位置，与注意力机制形成功能互补。

6.2 数学公式

- 前馈网络FFN(FeedForward Network)的计算过程可表示为：

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2 \tag{10}$$

在标准实现中：

  - 第一层将维度从d_model扩展到d_ff
  - 第二层将维度从d_ff恢复为d_model

- 实际实现中常省略偏置项（bias=False）

带Dropout的改进公式：

$$\mathrm{FFN}(x) = \mathrm{Dropout}(\mathrm{ReLU}(xW_1)W_2) \tag{11}$$

6.3 代码实现

- 前馈网络模块实现

```python
import torch
from torch import nn

class FeedForward(nn.Module):
    def __init__(self, d_model, ffn_hidden, dropout_prob=0.1):
        """
        :param ffn_hidden: 中间隐藏层维度
        """
        super(FeedForward, self).__init__()
        self.ffn = nn.Sequential(
            nn.Linear(in_features=d_model, out_features=ffn_hidden, bias=False),
            nn.ReLU(),
            nn.Linear(in_features=ffn_hidden, out_features=d_model, bias=False),
            nn.Dropout(dropout_prob)
        )

    def forward(self, x):
        output = self.ffn(x)
        return output
```

- 参数说明

参数名称	说明	典型值
d_model	输入/输出维度(embedding维度)	512
ffn_hidden	中间层扩展维度	2048
dropout_prob	Dropout概率（输出层之后应用）	0.1

- 数据流经各层的形状变化：

操作步骤	张量形状变化示例
输入数据	[batch_size, seq_len, d_model]
第一线性层（d_model→ffn_hidden）	[batch_size, seq_len, ffn_hidden]
ReLU激活	[batch_size, seq_len, ffn_hidden]
第二线性层（ffn_hidden→d_model）	[batch_size, seq_len, d_model]
Dropout	[batch_size, seq_len, d_model]

6.4 使用示例

- 测试代码

```python
if __name__ == "__main__":
    batch_size = 4
    seq_len = 100
    d_model = 512
    ffn_hidden = 2048

    x = torch.randn(batch_size, seq_len, d_model)
    ffn = FeedForward(d_model=d_model, ffn_hidden=ffn_hidden)
    out = ffn(x)

    print("输入形状:", x.shape)      # torch.Size([4, 100, 512])
    print("输出形状:", out.shape)    # torch.Size([4, 100, 512])
    print("输出均值:", out.mean().item())  # ≈0（受初始化影响）
```

7 五、Layer Normalization

7.1 层归一化介绍

- 层归一化（LayerNorm）通过对同一层的神经元输出进行归一化处理，有效提升模型训练稳定性。与BatchNorm不同，LayerNorm对单个样本的所有特征维度进行归一化，使其对序列长度变化具有更好的适应性。

  当处理变长序列数据时，LayerNorm保持每个时间步的独立计算特性，避免不同序列长度带来的统计量偏差。该操作通过可学习的缩放参数gamma和平移参数beta保留模型的表达能力。

7.2 数学公式

- 层归一化对输入张量最后一个维度进行标准化处理：

$$
\begin{aligned}
\mu &= \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} x_i \\
\sigma^2 &= \frac{1}{d_{model}} \sum_{i=1}^{d_{model}} (x_i - \mu)^2 \\
\text{out} &= \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta
\end{aligned}
\tag{12}
$$

  其中：

  - $\gamma \in \mathbb{R}^{d_model}$：可学习缩放参数（初始化为1）
  - $\beta \in \mathbb{R}^{d_model}$：可学习平移参数（初始化为0）
  - ϵ：数值稳定系数（默认1e-12）

  d_model 为模型维度。

7.3 代码实现

- 层归一化代码实现

```python
import torch
from torch import nn

class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-12):
        super(LayerNorm, self).__init__()
        self.d_model = d_model
        self.eps = eps

        # 可学习参数初始化
        self.gamma = nn.Parameter(torch.ones(d_model))  # 缩放参数
        self.beta = nn.Parameter(torch.zeros(d_model))   # 平移参数

    def forward(self, x):
        """
        shape of x:              [batch_size,seq_len,d_model]
```

```
        shape of mean and var :  [batch_size,seq_len,1]
        shape of gamma and beta: [d_model]
        """
        # 步骤1：计算最后一个维度的均值和方差
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1,
                    unbiased=False,  # 使用有偏方差估计（与PyTorch官方实现保持一致）
                    keepdim=True # 保持维度对其
                    )

        # 步骤2：标准化计算
        normalized = (x - mean) / torch.sqrt(var + self.eps)

        # 步骤3：仿射变换
        out = self.gamma * normalized + self.beta
        return out
```

- 注意
  1. 在统计学中，"unbiased"（无偏）通常指的是一个估计量，其期望值等于所估计的参数值。对于方差的计算，有两种常用的方法：无偏估计和最大似然估计。

     无偏估计（Bessel's Correction）：在计算样本方差时，通常使用无偏估计，其公式为：

     $$\mathrm{Var} = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2 \tag{13}$$

     这里我们除以的是 $N-1$，而不是样本数量 N。这是因为用 $N-1$ 除法能够补偿样本方差相对于总体方差的系统性低估，也就是所谓贝塞尔校正（Bessel's correction）。

     有偏估计（或最大似然估计，MLE）：这是普遍用于深度学习中的方法，就是直接使用：

     $$\mathrm{Var} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2 \tag{14}$$

     其中 N 为样本总量，延伸到我们transformer中就是 d_model 。
  2. 使用有偏估计可以减小计算的偏差，提升计算的效率和速度，这个同事也在深度学习过程中允许更快的梯度更新，虽然有轻微的偏差，但通常在大量数据下这并不显著影响模型的训练效果。
  3. `unbiased=False` 是指定要使用有偏估计来计算方差。这默认为 `False`，偏向于加速计算并使得在深度学习环境下的表现更加稳定。
- **维度计算流程**

操作步骤	张量形状变化示例
输入数据	[batch_size, seq_len, d_model]
步骤1 - 计算均值（dim=-1）	[batch_size, seq_len, 1]
步骤1 - 计算方差（dim=-1）	[batch_size, seq_len, 1]
步骤2 - 标准化计算	[batch_size, seq_len, d_model]
步骤3 - 线性变换（gamma/beta）	[batch_size, seq_len, d_model]

7.4 使用示例

- 测试代码

```python
if __name__ == "__main__":
    # 参数配置
    batch_size = 4
    seq_len = 100
    d_model = 512

    # 生成测试数据
    x = torch.randn(batch_size, seq_len, d_model)

    # 实例化层归一化模块
    layer_norm = LayerNorm(d_model=d_model)

    # 前向传播
    out = layer_norm(x)

    # 验证输出
    print("输入形状:", x.shape)        # torch.Size([4, 100, 512])
    print("输出形状:", out.shape)      # torch.Size([4, 100, 512])
    print("参数gamma形状:", layer_norm.gamma.shape)  # torch.Size([512])
    print("参数beta形状:", layer_norm.beta.shape)    # torch.Size([512])
```

8 五、RMSNorm（Root Mean Square Layer Normalization）

8.1 RMSNorm介绍

- RMSNorm（均方根层归一化）通过对神经元输出的均方根值进行缩放，在保持训练稳定性的同时降低计算复杂度。与LayerNorm相比，RMSNorm移除了均值中心化操作，仅通过均方根值进行缩放，减少了30%的计算量，在Transformer等模型中展现出相似的收敛效果。

  该归一化方法通过单一缩放参数gamma调整特征幅度，避免对输入进行平移操作。实验表明，RMSNorm在保持模型性能的同时，对计算资源的需求更低，尤其适合大规模模型训练。

8.2 数学公式

- RMSNorm对输入张量最后一个维度进行均方根标准化：

$$
\begin{aligned}
\mathrm{RMS}(x) &= \sqrt{\frac{1}{d_model}\sum_{i=1}^{d_model} x_i^2} \\
\mathrm{out} &= \gamma \cdot \frac{x}{\sqrt{\mathrm{RMS}(x)^2 + \epsilon}}
\end{aligned}
\tag{15}
$$

  其中：

  - $\gamma \in \mathbb{R}^{d_model}$：可学习缩放参数（初始化为1）
  - ϵ：数值稳定系数（一个很小的值，比如1e-12）

  d_model 为特征维度。

8.3 代码实现

- RMSNorm代码实现

```python
class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-8):
        super().__init__()
        self.d_model = d_model
        self.eps = eps

        # 可学习参数初始化
        self.gamma = nn.Parameter(torch.ones(d_model))  # 缩放参数

    def forward(self, x):
        """
        shape of x:             [batch_size, seq_len, d_model]
        shape of gamma:         [d_model]
        """
        # 步骤1：计算均方根 [batch_size, seq_len, 1]
```

```
        rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + self.eps)

        # 步骤2：均方根标准化计算  [batch_size, seq_len, d_model]
        normalized = x / rms

        # 步骤3：缩放变换          [batch_size, seq_len, d_model]
        out = self.gamma * normalized
        return out
```

- 维度计算流程

操作步骤	张量形状变化示例
输入数据 x	[batch_size, seq_len, d_model]
步骤1 - 计算平方均值（dim=-1）	[batch_size, seq_len, 1]
步骤2 - 均方根标准化	[batch_size, seq_len, d_model]
步骤3 - 缩放变换	[batch_size, seq_len, d_model]

8.4 使用示例

- 测试代码

```
if __name__ == "__main__":
    # 参数配置
    batch_size = 4
    seq_len = 100
    d_model = 512

    # 生成测试数据
    x = torch.randn(batch_size, seq_len, d_model)

    # 实例化RMSNorm模块
    rms_norm = RMSNorm(d_model=d_model)

    # 前向传播
    out = rms_norm(x)

    # 验证输出
    print("输入形状:", x.shape)          # torch.Size([4, 100, 512])
    print("输出形状:", out.shape)        # torch.Size([4, 100, 512])
    print("参数gamma形状:", rms_norm.gamma.shape)  # torch.Size([512])
```

五、RMSNorm（Root Mean Square Layer Normalization）

8.5 RMSNorm介绍

- RMSNorm（均方根层归一化）通过对神经元输出的均方根值进行缩放，在保持训练稳定性的同时降低计算复杂度。与LayerNorm相比，RMSNorm移除了均值中心化操作，仅通过均方根值进行缩放，减少了30%的计算量，在Transformer等模型中展现出相似的收敛效果。

该归一化方法通过单一缩放参数gamma调整特征幅度，避免对输入进行平移操作。实验表明，RMSNorm在保持模型性能的同时，对计算资源的需求更低，尤其适合大规模模型训练。

8.6 数学公式

- RMSNorm对输入张量最后一个维度进行均方根标准化：

$$
\begin{aligned}
\mathrm{RMS}(x) &= \sqrt{\frac{1}{d_model}\sum_{i=1}^{d_model} x_i^2} \\
\mathrm{out} &= \gamma \cdot \frac{x}{\sqrt{\mathrm{RMS}(x)^2 + \epsilon}}
\end{aligned}
\tag{16}
$$

  其中：

  - $\gamma \in \mathbb{R}^{d_model}$：可学习缩放参数（初始化为1）
  - ϵ：数值稳定系数（一个很小的值，比如1e-12）

  d_model 为特征维度。

8.7 代码实现

- RMSNorm代码实现

```python
class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-8):
        super().__init__()
        self.d_model = d_model
        self.eps = eps

        # 可学习参数初始化
        self.gamma = nn.Parameter(torch.ones(d_model))  # 缩放参数

    def forward(self, x):
        """
        shape of x:             [batch_size, seq_len, d_model]
        shape of gamma:         [d_model]
        """
        # 步骤1：计算均方根 [batch_size, seq_len, 1]
        rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + self.eps)

        # 步骤2：均方根标准化计算  [batch_size, seq_len, d_model]
        normalized = x / rms

        # 步骤3：缩放变换         [batch_size, seq_len, d_model]
        out = self.gamma * normalized
        return out
```

- 维度计算流程

操作步骤	张量形状变化示例
输入数据 x	[batch_size, seq_len, d_model]
步骤1 - 计算平方均值（dim=-1）	[batch_size, seq_len, 1]
步骤2 - 均方根标准化	[batch_size, seq_len, d_model]
步骤3 - 缩放变换	[batch_size, seq_len, d_model]

8.8 使用示例

- 测试代码

```python
if __name__ == "__main__":
    # 参数配置
    batch_size = 4
    seq_len = 100
    d_model = 512

    # 生成测试数据
    x = torch.randn(batch_size, seq_len, d_model)

    # 实例化RMSNorm模块
    rms_norm = RMSNorm(d_model=d_model)

    # 前向传播
    out = rms_norm(x)

    # 验证输出
    print("输入形状:", x.shape)          # torch.Size([4, 100, 512])
    print("输出形状:", out.shape)        # torch.Size([4, 100, 512])
    print("参数gamma形状:", rms_norm.gamma.shape)  # torch.Size([512])
```

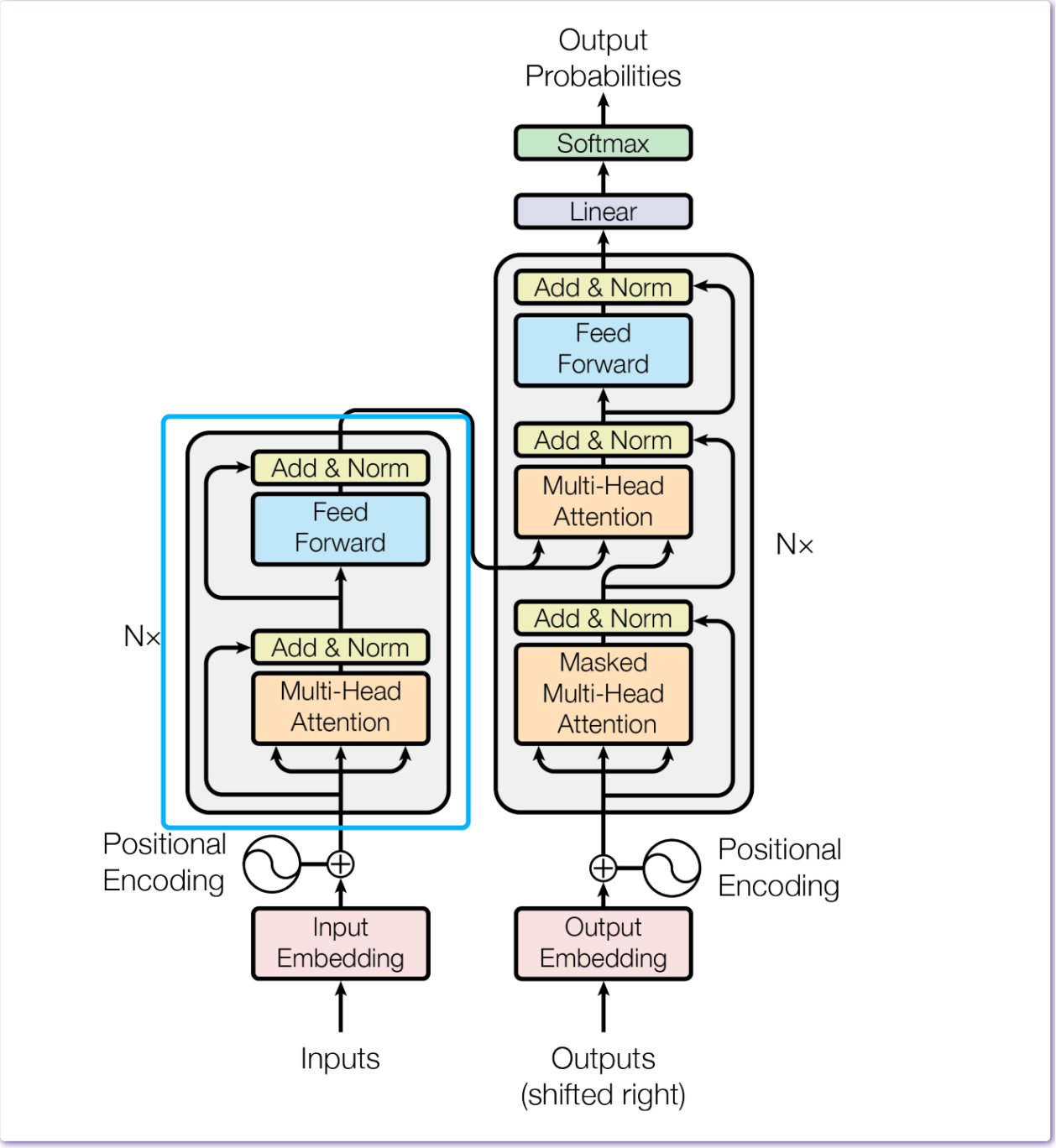
9 六、Encoder Layer

- **论文地址**

> https://arxiv.org/pdf/1706.03762

9.1 Encoder层介绍

- Encoder层是Transformer编码器的核心组件，由多头自注意力和前馈神经网络两个子层构成。每个子层都包含残差连接（Residual Connection）和层归一化（Layer Normalization），有效缓解梯度消失问题并加速训练。



今天这里实现的是上图中蓝色框中的单层EncoderLayer，不包含 embedding和位置编码。

主要处理流程：

1. **多头自注意力机制**：建立序列内部元素之间的依赖关系
2. **前馈神经网络**：进行非线性特征变换
3. **残差连接 + 层归一化**：在两个子层后各执行一次
4. **Dropout**：最终输出前进行随机失活

9.2 数学表达

- 子层处理流程公式化表示：

$$\text{LayerOutput} = \text{LayerNorm}(x + \text{Sublayer}(x)) \tag{17}$$

对于Encoder层具体展开：

$$\begin{aligned} z &= \text{LayerNorm}(x + \text{MultiHeadAttention}(x)) \\ out &= \text{LayerNorm}(z + \text{FeedForward}(z)) \end{aligned} \tag{18}$$

9.3 代码实现

- 其他层的实现

层名	链接
PositionEncoding	https://blog.csdn.net/hbkybkzw/article/details/147431820
calculate_attention	https://blog.csdn.net/hbkybkzw/article/details/147462845
MultiHeadAttention	https://blog.csdn.net/hbkybkzw/article/details/147490387
FeedForward	https://blog.csdn.net/hbkybkzw/article/details/147515883
LayerNorm	https://blog.csdn.net/hbkybkzw/article/details/147516529

下面统一在before.py中导入

- 实现单层的encoder层

```python
import torch
from torch import nn

from before import PositionEncoding,calculate_attention,MultiHeadAttention,FeedForward,LayerNorm

class EncoderLayer(nn.Module):
    """Transformer编码器层"""

    def __init__(self, n_heads, d_model, ffn_hidden, dropout_prob=0.1):
        super(EncoderLayer, self).__init__()
        self.self_multi_head_att = MultiHeadAttention(n_heads, d_model, dropout_prob)
        self.ffn = FeedForward(d_model, ffn_hidden, dropout_prob)
        self.norm1 = LayerNorm(d_model)
```

```python
        self.norm2 = LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout_prob)

    def forward(self, x, mask=None):
        # 自注意力子层
        _x = x  # 原始输入保存用于残差连接
        att_out = self.self_multi_head_att(x, x, x, mask)
        att_out += _x  # 残差连接
        att_out = self.norm1(att_out)  # 层归一化

        # 前馈子层
        _x = att_out
        ffn_out = self.ffn(att_out)
        ffn_out += _x  # 残差连接
        ffn_out = self.norm2(ffn_out)  # 层归一化

        return self.dropout(ffn_out)  # 最终输出
```

- 维度变化

处理阶段	张量形状示例
输入数据	[batch_size, seq_len, d_model]
自注意力输出（不改变维度）	[batch_size, seq_len, d_model]
残差连接+层归一化	[batch_size, seq_len, d_model]
前馈网络输出（不改变维度）	[batch_size, seq_len, d_model]
最终输出	[batch_size, seq_len, d_model]

9.4 使用示例

- 测试用例

```python
if __name__ == "__main__":
    # 模拟输入：batch_size=4，序列长度100，维度512
    x = torch.randn(4, 100, 512)

    # 实例化编码器层：8头注意力，512维，前馈层2048维，10% dropout
    encoder_layer = EncoderLayer(n_heads=8, d_model=512, ffn_hidden=2048, dropout_prob=0.1)

    # 前向传播
    out = encoder_layer(x, mask=None)

    print("Input shape:", x.shape)    # torch.Size([4, 100, 512])
    print("Output shape:", out.shape) # torch.Size([4, 100, 512])
```

---

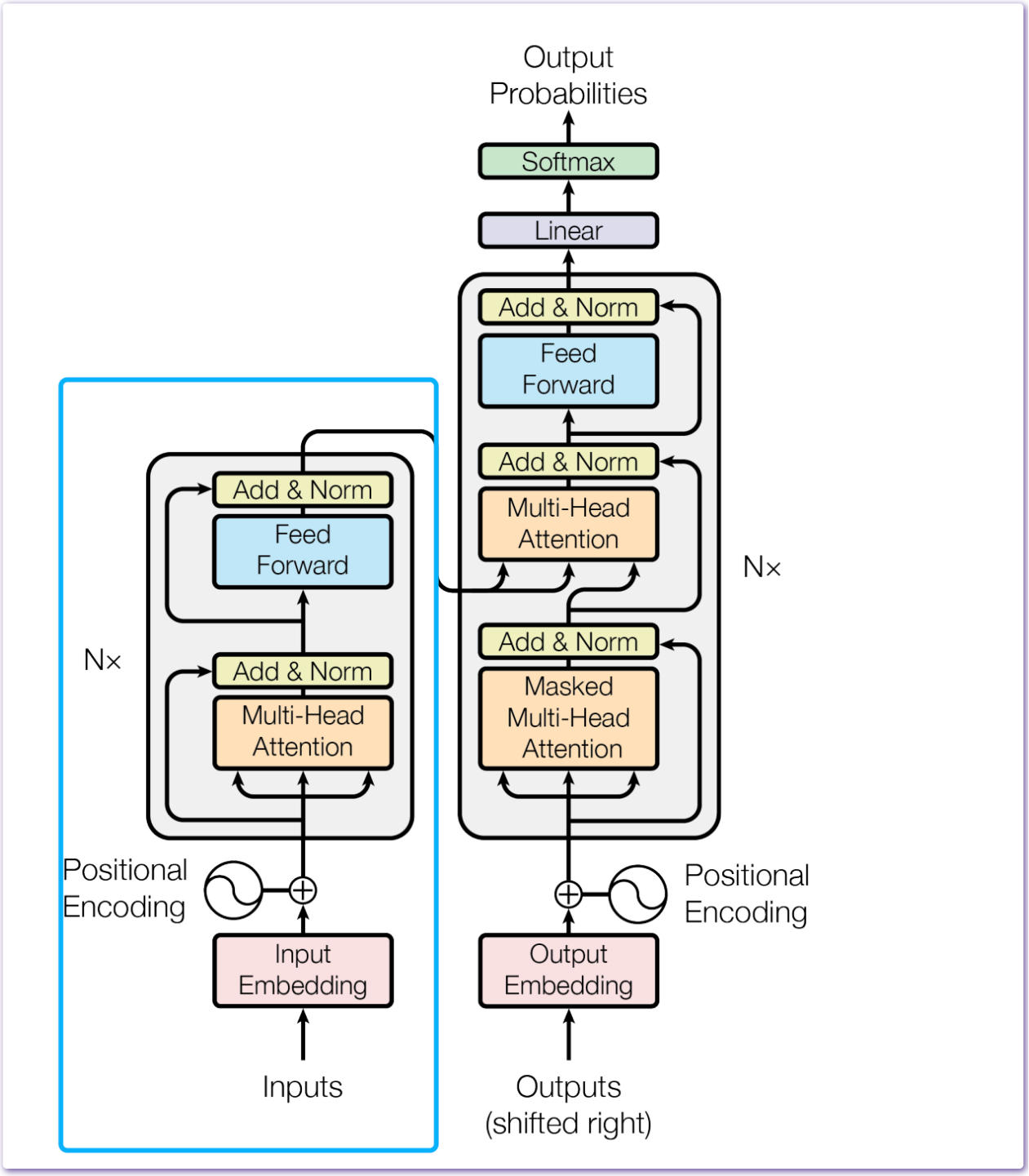
10 七、Encoder

- **论文地址**

> https://arxiv.org/pdf/1706.03762

10.1 Encoder结构介绍

- Transformer Encoder是Transformer模型的核心组件，负责对输入序列进行特征提取和语义编码。通过堆叠多层结构相同的编码层（Encoder Layer），每层包含自注意力机制和前馈神经网络，实现对输入信息的逐层抽象。



这里实现上图蓝色框中的整个Encoder结构

主要组件包含：

1. **词嵌入层**：将离散的token索引映射为连续向量
2. **位置编码**：注入序列的位置信息
3. **多层编码层**：通过多头注意力和前馈网络进行特征变换

10.2 数学流程

- 编码器的计算过程可形式化为：

$$\begin{aligned} \text{Embedding}: X_{embed} &= \text{Embedding}(X_{input}) \\ \text{PositionEncode}: X_{pos} &= X_{embed} + \text{PositionEncoding}(seq_len) \\ \text{EncoderLayers}: H_{out} &= \text{LayerNorm}(\text{FFN}(\text{MHA}(X_{pos}) + X_{pos}) \end{aligned} \tag{19}$$

其中每个编码层包含：

1. 多头自注意力（Multi-Head Self-Attention）
2. 残差连接 + 层归一化
3. 前馈神经网络（Feed-Forward Network）
4. 二次残差连接 + 层归一化

10.3 代码实现

- 其他层的实现

层名	链接
PositionEncoding	https://blog.csdn.net/hbkybkzw/article/details/147431820
calculate_attention	https://blog.csdn.net/hbkybkzw/article/details/147462845
MultiHeadAttention	https://blog.csdn.net/hbkybkzw/article/details/147490387
FeedForward	https://blog.csdn.net/hbkybkzw/article/details/147515883
LayerNorm	https://blog.csdn.net/hbkybkzw/article/details/147516529
EncoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147591824

下面统一在before.py中导入

- 实现 transformer 编码器 Encoder

```
import torch
from torch import nn

from before import PositionEncoding,calculate_attention,MultiHeadAttention,FeedForward,LayerNorm, EncoderLayer

```

```python

class Encoder(nn.Module):
    def __init__(self, vocab_size, padding_idx, d_model, n_heads, ffn_hidden,
                 dropout_prob=0.1, num_layers=6, max_seq_len=512):
        super(Encoder, self).__init__()
        # 词嵌入层（含padding掩码）
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=d_model,
            padding_idx=padding_idx
        )
        # 位置编码器
        self.position_encode = PositionEncoding(
            d_model=d_model,
            max_seq_len=max_seq_len
        )
        # 编码层堆叠
        self.encode_layers = nn.ModuleList([
            EncoderLayer(
                n_heads=n_heads,
                d_model=d_model,
                ffn_hidden=ffn_hidden,
                dropout_prob=dropout_prob
            ) for _ in range(num_layers)
        ])

    def forward(self, x, src_mask=None):
        # 输入形状：[batch_size, seq_len]
        embed_x = self.embedding(x)  # 词向量映射
        pos_encode_x = self.position_encode(embed_x)  # 位置编码

        # 逐层处理
        for layer in self.encode_layers:
            pos_encode_x = layer(pos_encode_x, mask=src_mask)

        return pos_encode_x  # 输出形状：[batch_size, seq_len, d_model]
```

关键组件说明

1. 词嵌入层（Embedding）：将离散的token索引转换为d_model维连续向量，自动处理padding位置的零值屏蔽
2. 位置编码（PositionEncoding）：使用正弦/余弦函数生成位置特征，与词向量相加实现位置信息注入
3. 编码层堆叠（encode_layers）：每个编码层包含：多头自注意力机制；前馈神经网络；残差连接；层归一化

- 维度变化

处理阶段	张量形状变化示例
原始输入	[batch_size, seq_len]
词嵌入层输出	[batch_size, seq_len, d_model]
位置编码后	[batch_size, seq_len, d_model]
每个编码层输出	[batch_size, seq_len, d_model]

10.4 使用示例

- 测试用例

```python
if __name__ == "__main__":
    # 模拟输入：batch_size=4，序列长度64
    x = torch.randint(0, 100, (4, 64))

    # 实例化编码器：词表100，模型维度512，8头，2048前馈维度，8层
    encoder = Encoder(
        vocab_size=100,
        padding_idx=0,
        d_model=512,
        n_heads=8,
        ffn_hidden=2048,
        num_layers=8,
        max_seq_len=512
    )

    # 前向传播
    out = encoder(x)

    print("输入形状:", x.shape)     # torch.Size([4, 64])
    print("输出形状:", out.shape)   # torch.Size([4, 64, 512])
```

- 上述参数解释

参数名	典型值	作用描述
vocab_size	30000	词表大小
padding_idx	0	指定padding token的索引
d_model	512	模型隐藏层维度
n_heads	8	注意力头数量
ffn_hidden	2048	前馈网络中间层维度
num_layers	6/8	编码层堆叠数量
max_seq_len	512	支持的最大序列长度

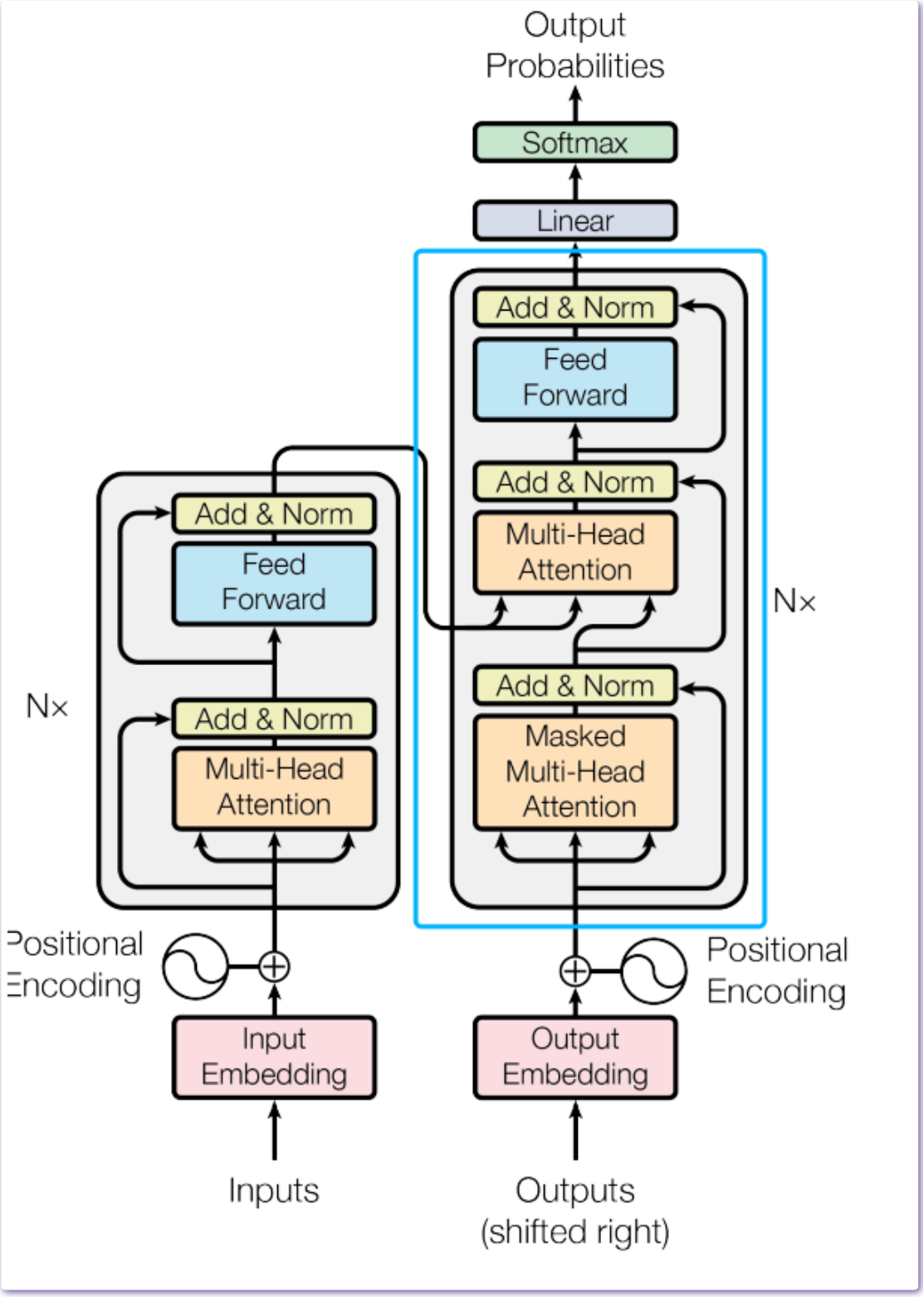
11 八、Decoder Layer

- **论文地址**

> **https://arxiv.org/pdf/1706.03762**

11.1 解码器层结构

- Transformer解码器层由三种核心组件构成：
    1. **Masked多头自注意力**：关注解码器序列当前位置之前的上下文（因果掩码）
    2. **Encoder-Decoder多头注意力**：关注编码器输出的相关上下文
    3. **前馈神经网络**：进行非线性特征变换



今天这里实现的是上图中蓝色框中的单层DecoderLayer，不包含 embedding和位置编码,以及最后的Linear和Softmax。

主要处理流程：

1. **Decoder 的Masked自注意力**
2. **Encoder-Decoder自注意力**
3. **前馈神经网络**：进行非线性特征变换
4. **残差连接 + 层归一化**
5. **Dropout**：最终输出前进行随机失活

11.2 数学表达

- 解码器层计算过程分为三个阶段：
    1. **Masked自注意力阶段**：

$$\text{MaskedAtt}(Q,K,V) = \text{LayerNorm}(\text{MultiHead}(Q,K,V) + Residual) \tag{20}$$

    2. **Encoder-Decoder注意力阶段**：

$$\text{CrossAtt}(Q_{dec},K_{enc},V_{enc}) = \text{LayerNorm}(\text{MultiHead}(Q_{dec},K_{enc},V_{enc}) + Residual) \tag{21}$$

    3. **前馈网络阶段**：

$$\text{FFN}(x) = \text{LayerNorm}(\text{ReLU}(xW_1 + b_1)W_2 + b_2 + x) \tag{22}$$

  其中：
    1. d_model 为模型维度
    2. Residual 为残差连接
    3. 下标dec来源于Decoder自己的输出，下标enc为Encoder的输出

11.3 代码实现

- 实现单层

  其他层的实现

层名	链接
PositionEncoding	https://blog.csdn.net/hbkybkzw/article/details/147431820
calculate_attention	https://blog.csdn.net/hbkybkzw/article/details/147462845
MultiHeadAttention	https://blog.csdn.net/hbkybkzw/article/details/147490387
FeedForward	https://blog.csdn.net/hbkybkzw/article/details/147515883
LayerNorm	https://blog.csdn.net/hbkybkzw/article/details/147516529
EncoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147591824

  下面统一在before.py中导入

- 实现单层的DecoderLayer

```
import torch
from torch import nn

from before import PositionEncoding,calculate_attention,MultiHeadAttention,FeedForward,LayerNorm


class DecoderLayer(nn.Module):
```

```python
    def __init__(self, n_heads, d_model, ffn_hidden, dropout_prob=0.1):
        super(DecoderLayer, self).__init__()
        self.masked_att = MultiHeadAttention(n_heads=n_heads, d_model=d_model, dropout_prob=dropout_prob)
        self.att = MultiHeadAttention(n_heads=n_heads, d_model=d_model, dropout_prob=dropout_prob)
        self.norms = nn.ModuleList([LayerNorm(d_model=d_model) for _ in range(3)])  # 三个归一化层
        self.ffn = FeedForward(d_model=d_model, ffn_hidden=ffn_hidden, dropout_prob=dropout_prob)
        self.dropout = nn.Dropout(dropout_prob)

    def forward(self, x, encoder_kv, dst_mask=None, src_dst_mask=None):
        # 第一阶段：Decoder 的Masked自注意力
        _x = x
        mask_att_out = self.masked_att(q=x, k=x, v=x, mask=dst_mask)
        mask_att_out = self.norms[0](mask_att_out + _x)  # 残差连接后归一化

        # 第二阶段：Encoder-Decoder注意力
        _x = mask_att_out
        att_out = self.att(q=mask_att_out, k=encoder_kv, v=encoder_kv, mask=src_dst_mask)
        att_out = self.norms[1](att_out + _x)

        # 第三阶段：前馈网络
        _x = att_out
        ffn_out = self.ffn(att_out)
        ffn_out = self.norms[2](ffn_out + _x)

        return self.dropout(ffn_out)
```

- 注意力掩码机制

掩码类型	作用域	功能描述
`dst_mask`	目标序列自注意力	防止当前位置关注未来信息（因果掩码）
`src_dst_mask`	编码器-解码器注意力	控制解码器查询对编码器键值对的访问权限

- 参数说明

参数名	类型	说明
`n_heads`	int	注意力头数量
`d_model`	int	模型隐藏层维度
`ffn_hidden`	int	前馈网络中间层维度（通常4倍）
`dropout_prob`	float	Dropout概率（默认0.1）

11.4 使用示例

- 测试代码

```python
if __name__ == "__main__":
    # 实例化解码器层：8头，512维，前馈层2048，20% dropout
    decoder_layer = DecoderLayer(n_heads=8, d_model=512, ffn_hidden=2048, dropout_prob=0.2)

    # 模拟输入：batch_size=4，目标序列长度50，编码器输出长度80
    x = torch.randn(4, 50, 512)
    encoder_out = torch.randn(4, 80, 512)

    tgt_mask = None
    src_mask = None

    output = decoder_layer(x, encoder_out, dst_mask=tgt_mask, src_dst_mask=src_mask)

    print("输入形状:", x.shape)
    print("encode_kv 形状:", encoder_out.shape)
    print("输出形状:", output.shape)
```

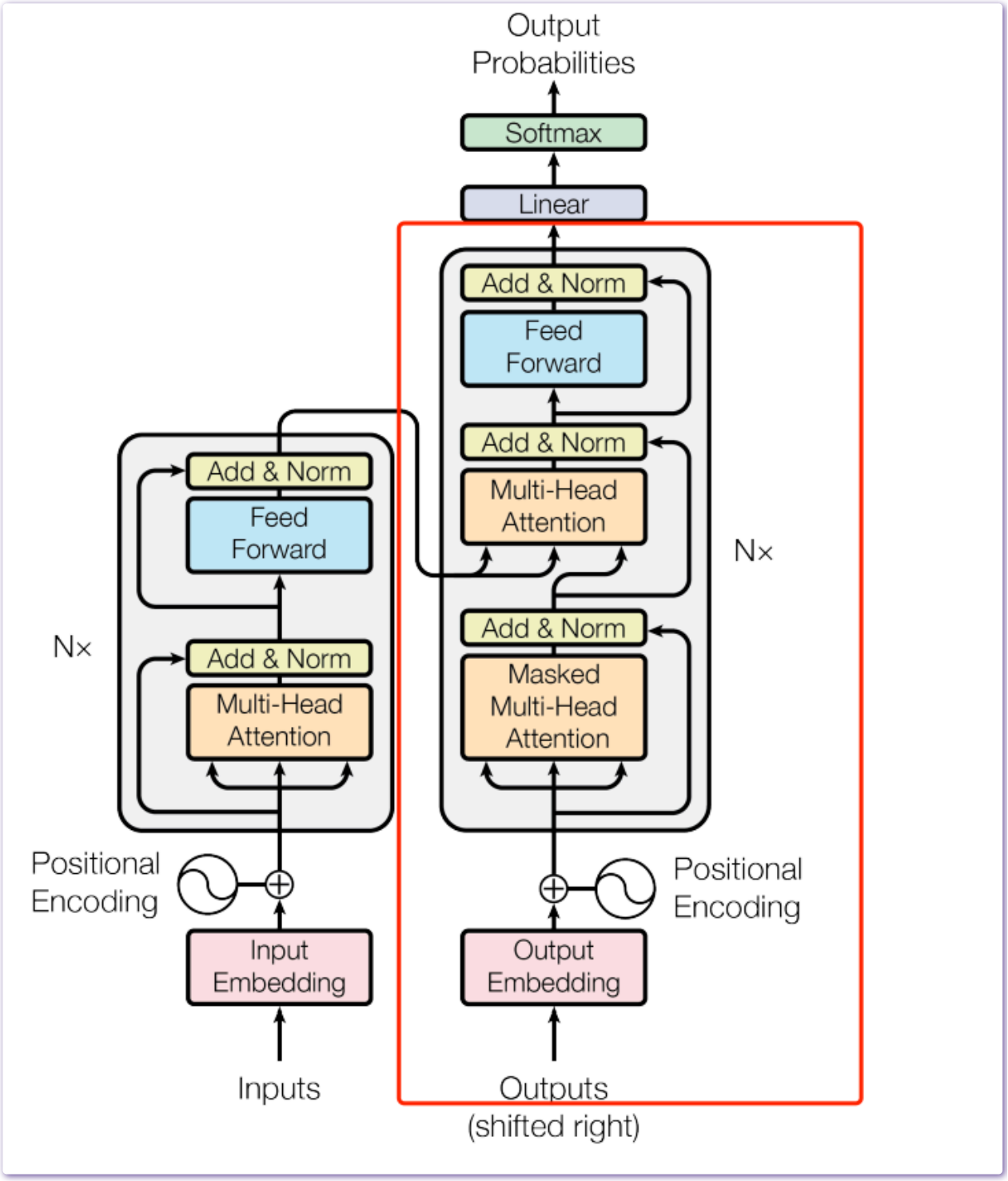
12 九、Decoder

- **论文地址**

> https://arxiv.org/pdf/1706.03762

12.1 Decoder结构介绍

- Transformer Decoder是Transformer模型的核心生成组件，负责基于编码器输出和已生成内容预测后续token。通过堆叠多层结构相同的解码层（Decoder Layer），每层包含自注意力机制、编码器-解码器注意力机制和前馈神经网络，实现序列生成的上下文感知。



实现上图红色框中的整个Decoder结构

主要组件包含：

1. **词嵌入层**：将目标序列token映射为连续向量
2. **位置编码**：加入自回归生成的位置信息
3. **多层解码层**：通过三类注意力机制进行上下文建模

12.2 数学流程

- 解码器的计算过程可形式化为：

$$\begin{aligned} \text{Embedding}: Y_{embed} &= \text{Embedding}(Y_{input}) \\ \text{PositionEncode}: Y_{pos} &= Y_{embed} + \text{PositionEncoding}(seq_len) \\ \text{DecoderLayers}: H_{out} &= \text{LayerNorm}(\text{FFN}(\text{EncDec-Attn}(\text{Self-Attn}(Y_{pos})))) \end{aligned} \tag{23}$$

每个解码层(DecoderLayers)包含：

1. 带掩码的多头自注意力（Masked Multi-Head Self-Attention）
2. 编码器-解码器注意力（Encoder-Decoder Attention）
3. 前馈神经网络（Feed-Forward Network）
4. 三次残差连接 + 层归一化

12.3 代码实现

- 其他层的实现

层名	链接
PositionEncoding	https://blog.csdn.net/hbkybkzw/article/details/147431820
calculate_attention	https://blog.csdn.net/hbkybkzw/article/details/147462845
MultiHeadAttention	https://blog.csdn.net/hbkybkzw/article/details/147490387
FeedForward	https://blog.csdn.net/hbkybkzw/article/details/147515883
LayerNorm	https://blog.csdn.net/hbkybkzw/article/details/147516529
EncoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147591824
Encoder	https://blog.csdn.net/hbkybkzw/article/details/147616115
DecoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147616556

下面统一在before.py中导入

- 实现 transformer 解码器 Decoder

```
import torch
from torch import nn

```

```python
from before import PositionEncoding,calculate_attention,MultiHeadAttention,FeedForward,LayerNorm,DecoderLayer

class Decoder(nn.Module):
    def __init__(self, vocab_size, padding_idx, d_model, n_heads, ffn_hidden,
                 dropout_prob=0.1, num_layers=6, max_seq_len=512):
        super(Decoder, self).__init__()
        # 词嵌入层（含padding掩码）
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=d_model,
            padding_idx=padding_idx
        )
        # 位置编码器
        self.position_encode = PositionEncoding(
            d_model=d_model,
            max_seq_len=max_seq_len
        )
        # 解码层堆叠
        self.decode_layers = nn.ModuleList([
            DecoderLayer(
                n_heads=n_heads,
                d_model=d_model,
                ffn_hidden=ffn_hidden,
                dropout_prob=dropout_prob
            ) for _ in range(num_layers)
        ])

    def forward(self, x, encoder_kv, dst_mask=None, src_dst_mask=None):
        # 输入形状：[batch_size, tgt_seq_len]
        embed_x = self.embedding(x)                # 词向量映射
        pos_encode_x = self.position_encode(embed_x)  # 位置编码

        # 逐层处理（携带编码器输出）
        for layer in self.decode_layers:
            pos_encode_x = layer(
                x=pos_encode_x,
                encoder_kv=encoder_kv,
                dst_mask=dst_mask,
                src_dst_mask=src_dst_mask
            )

        return pos_encode_x  # 输出形状：[batch_size, tgt_seq_len, d_model]
```

关键组件说明：

1. **双注意力机制**：每层包含自注意力和编码器-解码器注意力
2. **掩码控制**：
    - `dst_mask`：防止解码时看到未来信息（三角掩码）
    - `src_dst_mask`：控制编码器-解码器注意力范围
3. **信息融合**：将编码器输出作为Key/Value供解码器查询

- 维度变化

处理阶段	张量形状变化示例
原始输入	[batch_size, tgt_seq_len]
词嵌入层输出	[batch_size, tgt_seq_len, d_model]
位置编码	[batch_size, tgt_seq_len, d_model]
编码器输出	[batch_size, src_seq_len, d_model]
解码层输出	[batch_size, tgt_seq_len, d_model]

12.4 使用示例

- 测试用例

```python
if __name__ == "__main__":
    # 模拟输入：batch_size=4，目标序列长度32
    y = torch.randint(0, 100, (4, 32))
    # 编码器输出：batch_size=4，源序列长度64，维度512
    encoder_out = torch.randn(4, 64, 512)

    decoder = Decoder(vocab_size=100,padding_idx=0,d_model=512,n_heads=8,ffn_hidden=2048,num_layers=6,max_seq_len=51)

    out = decoder(y, encoder_out)

    print("输入形状:", y.shape)
    print("编码器输出形状:", encoder_out.shape)
    print("解码器输出形状:", out.shape)
```

- 参数解释

参数名	典型值	作用描述
vocab_size	30000	目标语言词表大小
padding_idx	0	指定padding token的索引
d_model	512	模型隐藏层维度
n_heads	8	注意力头数量
ffn_hidden	2048	前馈网络中间层维度
num_layers	6	解码层堆叠数量
max_seq_len	512	支持的最大序列长度
encoder_kv	-	编码器输出，用于交叉注意力计算
dst_mask	-	目标序列自注意力掩码（三角形）
src_dst_mask	-	源-目标序列交叉注意力掩码

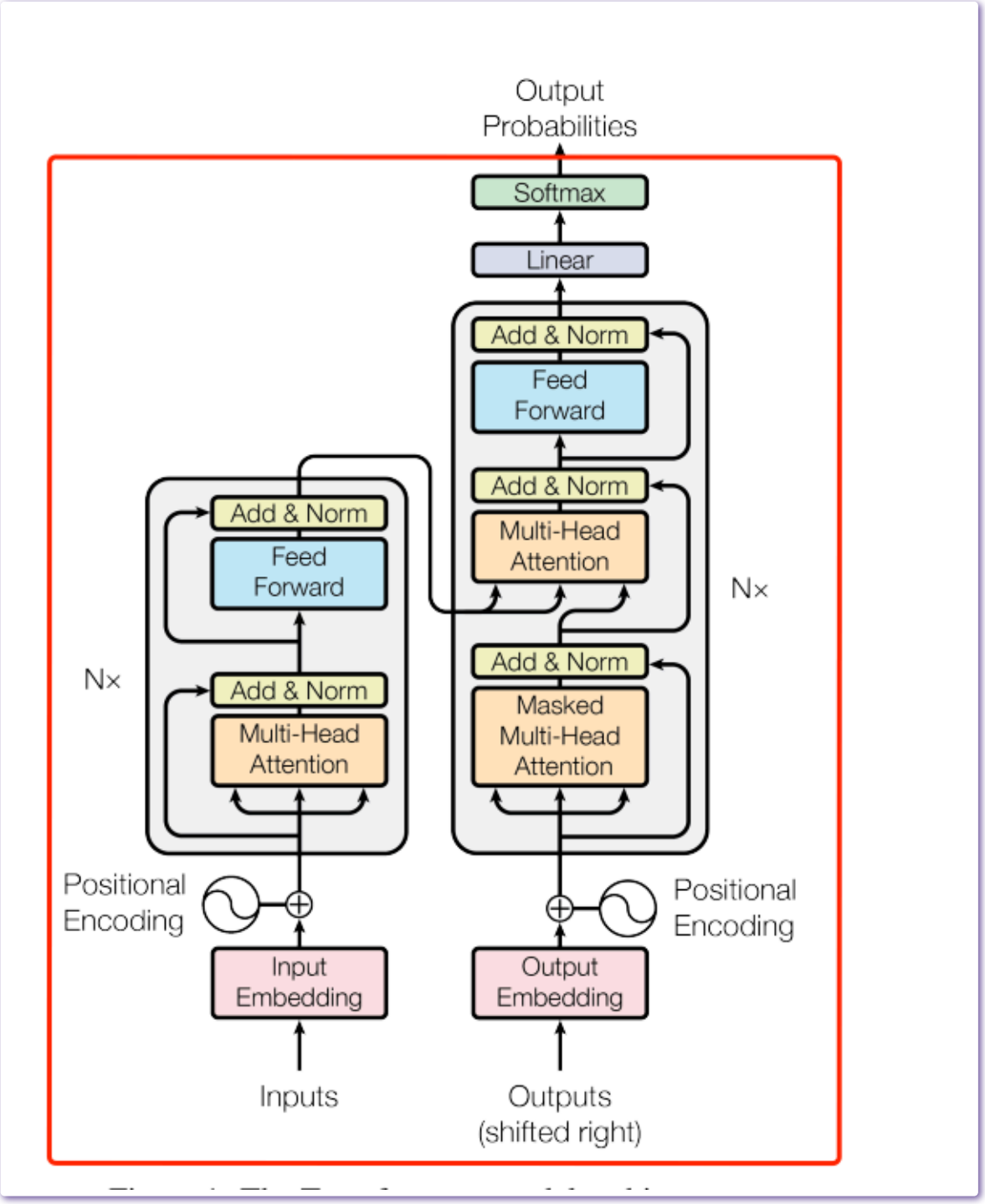
13 十、Transformer

- `https://arxiv.org/pdf/1706.03762`

> https://arxiv.org/pdf/1706.03762

13.1 Transformer结构介绍

- Transformer是一种基于自注意力机制的神经网络架构，用于序列到序列的转换任务。它由编码器(Encoder)和解码器(Decoder)两部分组成，通过自注意力机制实现对输入和输出序列的有效建模。



这里实现完整的Transformer结构，包含编码器和解码器

主要组件包含：

1. **编码器(Encoder)**：处理输入序列并生成上下文表示
2. **解码器(Decoder)**：基于编码器输出和已生成的序列预测下一个token
3. **线性层**：将解码器输出映射到目标词表空间

最后的softmax因为一般都包含在损失函数中计算所有可实现可不实现，不重要

13.2 数学流程

- Transformer的计算过程可形式化为：

$$\begin{aligned} \text{Encoder Output}: H_{enc} &= \text{Encoder}(X_{src}) \\ \text{Decoder Output}: H_{dec} &= \text{Decoder}(Y_{dst}, H_{enc}) \\ \text{Final Output}: P(y) &= \text{Softmax}(\text{Linear}(H_{dec})) \end{aligned} \tag{24}$$

其中：

1. 编码器处理源序列生成上下文表示
2. 解码器结合编码器输出和目标序列进行自回归生成
3. 线性层将解码器输出映射到词表维度，通过Softmax得到概率分布

13.3 代码实现

- 其他层的实现

层名	链接
PositionEncoding	https://blog.csdn.net/hbkybkzw/article/details/147431820
calculate_attention	https://blog.csdn.net/hbkybkzw/article/details/147462845
MultiHeadAttention	https://blog.csdn.net/hbkybkzw/article/details/147490387
FeedForward	https://blog.csdn.net/hbkybkzw/article/details/147515883
LayerNorm	https://blog.csdn.net/hbkybkzw/article/details/147516529
EncoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147591824
Encoder	https://blog.csdn.net/hbkybkzw/article/details/147616115
DecoderLayer	https://blog.csdn.net/hbkybkzw/article/details/147616556
Decoder	https://blog.csdn.net/hbkybkzw/article/details/147628354

下面统一在before.py中导入

- 实现 Transformer 模型

```
import torch
from torch import nn

```

```python
from before import Encoder, Decoder


class Transformer(nn.Module):
    def __init__(self, encode_vocab_size, decode_vocab_size, padding_idx, d_model,
                 n_heads, ffn_hidden, dropout_prob=0.1, num_layers=6, max_seq_len=512):
        super(Transformer, self).__init__()

        # 编码器：处理输入序列
        self.encoder = Encoder(
            vocab_size=encode_vocab_size,
            padding_idx=padding_idx,
            d_model=d_model,
            n_heads=n_heads,
            ffn_hidden=ffn_hidden,
            dropout_prob=dropout_prob,
            num_layers=num_layers,
            max_seq_len=max_seq_len
        )

        # 解码器：生成输出序列
        self.decoder = Decoder(
            vocab_size=decode_vocab_size,
            padding_idx=padding_idx,
            d_model=d_model,
            n_heads=n_heads,
            ffn_hidden=ffn_hidden,
            dropout_prob=dropout_prob,
            num_layers=num_layers,
            max_seq_len=max_seq_len
        )

        # 输出线性层：映射到目标词表空间
        self.linear = nn.Linear(in_features=d_model, out_features=decode_vocab_size)

        self.padding_idx = padding_idx

    def generate_mask(self, query, key, is_triu_mask=False):
        """
        生成注意力掩码矩阵
        shape of query,key: [batch_size,seq_len]
        """
        device = query.device
        batch, seq_q = query.shape
        _, seq_k = key.shape

        # 创建padding掩码
        mask = (key == self.padding_idx)
        mask = mask.unsqueeze(1).unsqueeze(2)  # [batch,1,1,seq_k]
        mask = mask.expand(batch, 1, seq_q, seq_k).to(device)

        # 创建前瞻掩码（解码器中使用）
        if is_triu_mask:
            dst_triu_mask = torch.triu(
```

```python
                torch.ones(seq_q, seq_k, dtype=torch.bool),
                diagonal=1  # 需要往右移动一个，因为第一个单词是开始字符
            )  # [seq_q,seq_k]
            dst_triu_mask = dst_triu_mask.unsqueeze(0).unsqueeze(1).expand(
                batch, 1, seq_q, seq_k
            ).to(device)
            return mask | dst_triu_mask  # 取并集

        return mask

    def forward(self, src, dst):
        """
        前向传播过程
        :param src: encoder部分的输入
        :param dst: decoder部分的输出
        """
        # 生成编码器的padding掩码
        src_mask = self.generate_mask(src, src)

        # 编码器前向传播
        encoder_out = self.encoder(src, src_mask=src_mask)

        # 生成解码器的掩码（包含padding掩码和前瞻掩码）
        dst_mask = self.generate_mask(dst, dst, is_triu_mask=True)

        # 生成编码器-解码器的掩码
        src_dst_mask = self.generate_mask(dst, src)

        # 解码器前向传播
        decode_out = self.decoder(
            dst,
            encoder_kv=encoder_out,
            dst_mask=dst_mask,
            src_dst_mask=src_dst_mask
        )

        # 线性层映射到词表空间
        output = self.linear(decode_out)

        return output
```

关键组件说明

1. **编码器(Encoder)**：处理输入序列，生成上下文表示
2. **解码器(Decoder)**：结合编码器输出和已生成序列，预测下一个token
3. **掩码生成(generate_mask)**：创建三种掩码：
    - 编码器padding掩码：屏蔽填充位置
    - 解码器padding+前瞻掩码：屏蔽填充位置和未来位置
    - 编码器-解码器掩码：屏蔽编码器输出中的填充位置
4. **线性层**：将解码器输出映射到目标词表维度

- 维度变化

处理阶段	张量形状变化示例
源序列输入	[batch_size, src_seq_len]
目标序列输入	[batch_size, dst_seq_len]
编码器输出	[batch_size, src_seq_len, d_model]
解码器输出	[batch_size, dst_seq_len, d_model]
线性层输出	[batch_size, dst_seq_len, decode_vocab_size]

13.4 使用示例

- 测试用例

```python
if __name__ == "__main__":
    # 模拟输入：batch_size=4，源序列长度64，目标序列长度32
    src = torch.randint(0, 1000, (4, 64))
    dst = torch.randint(0, 1000, (4, 32))

    # 实例化Transformer：源词表1000，目标词表1000，模型维度512，8头，2048前馈维度
    transformer = Transformer(
        encode_vocab_size=1000,
        decode_vocab_size=1000,
        padding_idx=0,
        d_model=512,
        n_heads=8,
        ffn_hidden=2048,
        num_layers=6
    )

    # 前向传播
    output = transformer(src, dst)

    # 输出形状检查
    print(f"输出形状：{output.shape}")  # 预期：[4, 32, 1000]
```

14 十一、需要注意的函数

- 在上述transformers实现过程中需要注意的函数

14.1 `reshape` vs `view`

- `reshape` 和 `view` 都用于改变张量（Tensor）的形状。主要区别在于：
    1. **`view`**：要求张量在内存中是**连续 (contiguous)** 的。它返回的是原始张量的一个视图 (view)，意味着新张量与原张量共享底层数据，修改其中一个会影响另一个。`view` 的效率更高，因为它不涉及数据复制。
    2. **`reshape`**：功能更强大，不要求张量必须是连续的。如果张量是连续的，它的行为与 `view` 类似，返回一个视图。如果不是连续的，`reshape` 会先调用 `contiguous()` 方法创建一个数据的副本，然后在新副本上操作，因此可能返回一个副本 (copy)。

参数名	参数解释	示例
`shape`	`(int...)` 或 `tuple`，定义了输出张量的新形状。可以使用 `-1` 来让 PyTorch 自动推断该维度的大小。	`(2, 3)` 或 `-1, 6`

- **何时使用?**
    1. 当你确定张量是连续的，并且希望保证无数据复制以提高性能时，使用 `view`。
    2. 如果不确定张量的内存布局，或者代码的健壮性比微小的性能差异更重要时，使用 `reshape` 更安全。
- 示例代码

```python
import torch

# 创建一个张量
x = torch.arange(12) # tensor([0, 1, 2, ..., 11])

# 使用 reshape
x_reshaped = x.reshape(3, 4)
print("Reshaped:\n", x_reshaped)
"""
Reshaped:
 tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
"""

# 使用 view
x_view = x.view(3, 4)
print("Viewed:\n", x_view)
"""
Viewed:
 tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
"""
```

```python

# 验证共享数据
x_view[0, 0] = 99
print("Original x after modifying view:", x) # x 的第一个元素也被改变
"""
Original x after modifying view: tensor([99,  1,  2,  3,  4,  5,  6,  7,  8, 9, 10, 11])
"""

# 演示非连续张量
y = torch.arange(12).reshape(3, 4)
y_transposed = y.t() # 转置操作使张量非连续
print("y_transposed is contiguous:", y_transposed.is_contiguous()) # False
"""
y_transposed is contiguous: False
"""

# y_transposed.view(3, 4) # 这会报错: view size is not compatible with input tensor's size and stride
y_reshaped_from_transposed = y_transposed.reshape(3, 4) # reshape 可以正常工作
print("Reshaped from transposed:\n", y_reshaped_from_transposed)
"""
Reshaped from transposed:
 tensor([[ 0,  4,  8,  1],
        [ 5,  9,  2,  6],
        [10,  3,  7, 11]])
"""

```

- 在使用transpose之后，为兼容后续操作，尤其view操作，`view()` 要求张量必须是连续的。因此，直接在 `transpose()` 的结果上调用 `view()` 会导致 `RuntimeError`。这是最常见的需要使用 `.contiguous()` 的场景。

```python
# 错误的做法
x.transpose(0, 1).view(...)  # 会报错!

# 正确的做法
x.transpose(0, 1).contiguous().view(...) # 能够正常工作
```

`reshape()` 函数更智能一些。它可以作用于非连续张量。如果它发现张量非连续，它会**隐式地**创建一个副本，等价于执行了 `.contiguous()`。所以 `x.transpose(0, 1).reshape(...)` 是一个更简洁的写法。

操作链	结果	常见用途
`tensor.transpose(...)`	返回一个**非连续**的视图，速度快，无数据复制。	改变维度顺序。
`tensor.transpose(...).contiguous()`	返回一个**连续**的**新张量**，有数据复制开销。	为后续需要连续内存的操作（如 `view`）做准备，或进行性能优化。
`tensor.transpose(...).reshape(...)`	等价于 `...contiguous().view()`，是更现代、便捷的写法。	在改变维度后重塑张量形状。

14.2 `register_buffer`

- `register_buffer` 是 `nn.Module` 的一个方法，用于注册一个不需要计算梯度、但又希望作为模型状态一部分的张量。这些张量会被保存在模型的 `state_dict` 中，并随模型一起移动到 CPU/GPU，但不会被优化器 (Optimizer) 更新。

参数名	参数解释	示例
`name`	`(str)` 要注册的 buffer 的名称。	`"fixed_mask"`
`tensor`	`(torch.Tensor or None)` 要注册的张量。如果为 `None`，则移除该 buffer。	`torch.ones(10, 10)`
`persistent`	`(bool, optional)`，默认为 `True`。如果为 `True`，该 buffer 会成为模型 `state_dict` 的一部分。如果为 `False`，则不会。	`False`

- **常见用途**：
    1. 保存固定的、非训练参数，如预定义的常量或掩码（mask）。
    2. 保存模型在前向传播过程中需要用到的、但不是可学习参数的统计数据，例如运行中的均值和方差（如 BatchNorm 中的 `running_mean` 和 `running_var`）。
- 示例代码

```python
import torch
import torch.nn as nn

class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        # 注册一个固定的常量 buffer
        self.register_buffer("my_fixed_const", torch.tensor(3.14))
        # 注册一个不会被保存到 state_dict 的 buffer
        self.register_buffer("temp_buffer", torch.randn(5), persistent=False)
        # 这是一个可学习的参数
        self.my_param = nn.Parameter(torch.randn(1, 5))

model = MyModel()

# 查看 state_dict，只会包含 persistent=True 的 buffer 和 parameter temp_buffer 不在其中
print(model.state_dict())
"""
OrderedDict({'my_param': tensor([[ 0.0149, -0.9527, -0.3346, -0.0720, 0.3591]]), 'my_fixed_const': tensor(3.1400)})
"""

# 移动模型到 GPU，buffer 也会被移动
# if torch.cuda.is_available():
#     model.to('cuda')
#     print(model.my_fixed_const.device) # cuda:0
```

14.3 `torch.tril`

- `torch.tril` 函数返回一个张量，其中包含输入二维矩阵或一批矩阵的**下三角部分 (lower triangle)**，主对角线及以上（或以下，取决于 `diagonal` 参数）的元素被置为零。`tril` 是 "triangle lower" 的缩写。

参数名	参数解释	示例
`input`	`(torch.Tensor)` 输入张量，必须至少是二维的。	`torch.ones(3, 3)`
`diagonal`	`(int, optional)`，默认为 `0`。指定被保留的对角线。`diagonal=0` 指的是主对角线；`diagonal > 0` 指的是主对角线上方的对角线；`diagonal < 0` 指的是主对角线下方的对角线。	`-1`，`0`，`1`

- 示例代码

```python
import torch

# 创建一个 4x4 的张量
x = torch.ones(4, 4) * 2
print("Original Tensor:\n", x)
"""
Original Tensor:
 tensor([
         [2., 2., 2., 2.],
         [2., 2., 2., 2.],
         [2., 2., 2., 2.],
         [2., 2., 2., 2.]
        ])
"""

# 获取主对角线及以下的下三角部分 (diagonal=0)
lower_tri = torch.tril(x)
print("\nLower Triangle (diagonal=0):\n", lower_tri)
"""
Lower Triangle (diagonal=0):
 tensor([
         [2., 0., 0., 0.],
         [2., 2., 0., 0.],
         [2., 2., 2., 0.],
         [2., 2., 2., 2.]
        ])
"""

# 获取主对角线下方的部分 (diagonal=-1)
lower_tri_below_diag = torch.tril(x, diagonal=-1)
print("\nLower Triangle (diagonal=-1):\n", lower_tri_below_diag)
"""
Lower Triangle (diagonal=-1):
 tensor([
         [0., 0., 0., 0.],
         [2., 0., 0., 0.],
         [2., 2., 0., 0.],
         [2., 2., 2., 0.]
        ])
"""

# 获取主对角线上方一条对角线及以下的部分 (diagonal=1)
```

```
lower_tri_above_diag = torch.tril(x, diagonal=1)
print("\nLower Triangle (diagonal=1):\n", lower_tri_above_diag)
"""
Lower Triangle (diagonal=1):
 tensor([
         [2., 2., 0., 0.],
         [2., 2., 2., 0.],
         [2., 2., 2., 2.],
         [2., 2., 2., 2.]
        ])
"""
```

14.4 `masked_fill` vs `masked_fill_`

- 这两个函数根据一个布尔掩码（boolean mask）来填充张量。当掩码中对应位置的元素为 `True` 时，就将原张量中该位置的元素替换为指定的值 `value`。
- **区别**：
  - `masked_fill(mask, value)`： **不进行原地操作 (out-of-place)**。它返回一个新的、被填充过的张量，原始张量保持不变。
  - `masked_fill_(mask, value)`： **进行原地操作 (in-place)**。它会直接修改原始张量，并返回修改后的张量。函数名末尾的下划线 `_` 是 PyTorch 中原地操作的通用约定。

参数名	参数解释	示例
`mask`	`(torch.BoolTensor)` 布尔张量。其形状必须能通过广播 (broadcasting) 机制匹配到原张量。	`torch.tensor([[True, False], [False, True]])`
`value`	`(float)` 用于填充的值。	`-1e9` (在注意力机制中常用来屏蔽未来的位置)

- 示例代码

```
import torch
import numpy as np

# 创建一个原始张量
x = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)

# 创建一个布尔掩码
mask = torch.tensor([
                      [True,  False, True],
                      [False, True,  False]
                    ])

# 1. 使用 masked_fill (非原地操作)
new_x = x.masked_fill(mask, -np.inf)
print("Original x (after masked_fill):\n", x) # 原始张量不变
print("New tensor (from masked_fill):\n", new_x)
"""
```

```
Original x (after masked_fill):
 tensor([
         [1., 2., 3.],
         [4., 5., 6.]
        ])

New tensor (from masked_fill):
 tensor([
         [-inf, 2., -inf],
         [4., -inf, 6.]
        ])
"""


# 2. 使用 masked_fill_ (原地操作)
print("\n--- In-place operation ---")
y = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
print("Original y:\n", y)
"""
Original y:
 tensor([
         [1., 2., 3.],
         [4., 5., 6.]
        ])
"""

y.masked_fill_(mask, -np.inf)
print("y after masked_fill_:\n", y) # y 本身被修改
"""
y after masked_fill_:
 tensor([
         [-inf, 2., -inf],
         [4., -inf, 6.]
        ])
"""
```

14.5 scaled_dot_product_attention

- `torch.nn.functional.scaled_dot_product_attention`
- 这是 PyTorch 2.0 中引入的一个高效、融合的函数，用于计算**缩放点积注意力 (Scaled Dot-Product Attention)**，它是 Transformer 架构的核心组件。该函数将查询（Query）、键（Key）、值（Value）作为输入，并一步到位地计算出注意力输出。相比手动实现，它经过了高度优化（例如使用 FlashAttention），速度更快，内存占用更少。

参数名	参数解释	示例
`query`	`(Tensor)` 查询张量。 形状通常为 `(N, ..., L, E)`，其中 N 是批量大小，L 是目标序列长度，E 是嵌入维度。	`torch.randn(32, 8, 100, 64)`
`key`	`(Tensor)` 键张量。 形状通常为 `(N, ..., S, E)`，其中 S 是源序列长度。	`torch.randn(32, 8, 120, 64)`
`value`	`(Tensor)` 值张量。 形状通常为 `(N, ..., S, Ev)`，其中 `Ev` 是值的维度。	`torch.randn(32, 8, 120, 64)`
`attn_mask`	`(Tensor, optional)` 注意力掩码。 可以是布尔掩码或浮点数掩码，用于阻止某些位置参与注意力计算。对于布尔掩码，`True` 表示**不被允许**注意到的位置。形状需要能广播到 `(N, ..., L, S)`。	`torch.ones(100, 120, dtype=torch.bool)`
`dropout_p`	`(float, optional)`，默认为 `0.0`。 在 softmax 之后的注意力权重上应用的 dropout 概率。	`0.1`
`is_causal`	`(bool, optional)`，默认为 `False`。 如果为 `True`，会自动应用一个因果掩码 (causal mask)，确保每个查询位置只能关注到它自己及之前的位置。这在自回归解码 (auto-regressive decoding) 中非常有用。 注意：当 `is_causal=True` 时，不能再提供 `attn_mask`。	`True`

- 计算公式：

$$\mathbf{Attention}(Q, K, V) = \mathbf{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \mathbf{attn_mask}\right)V \tag{25}$$

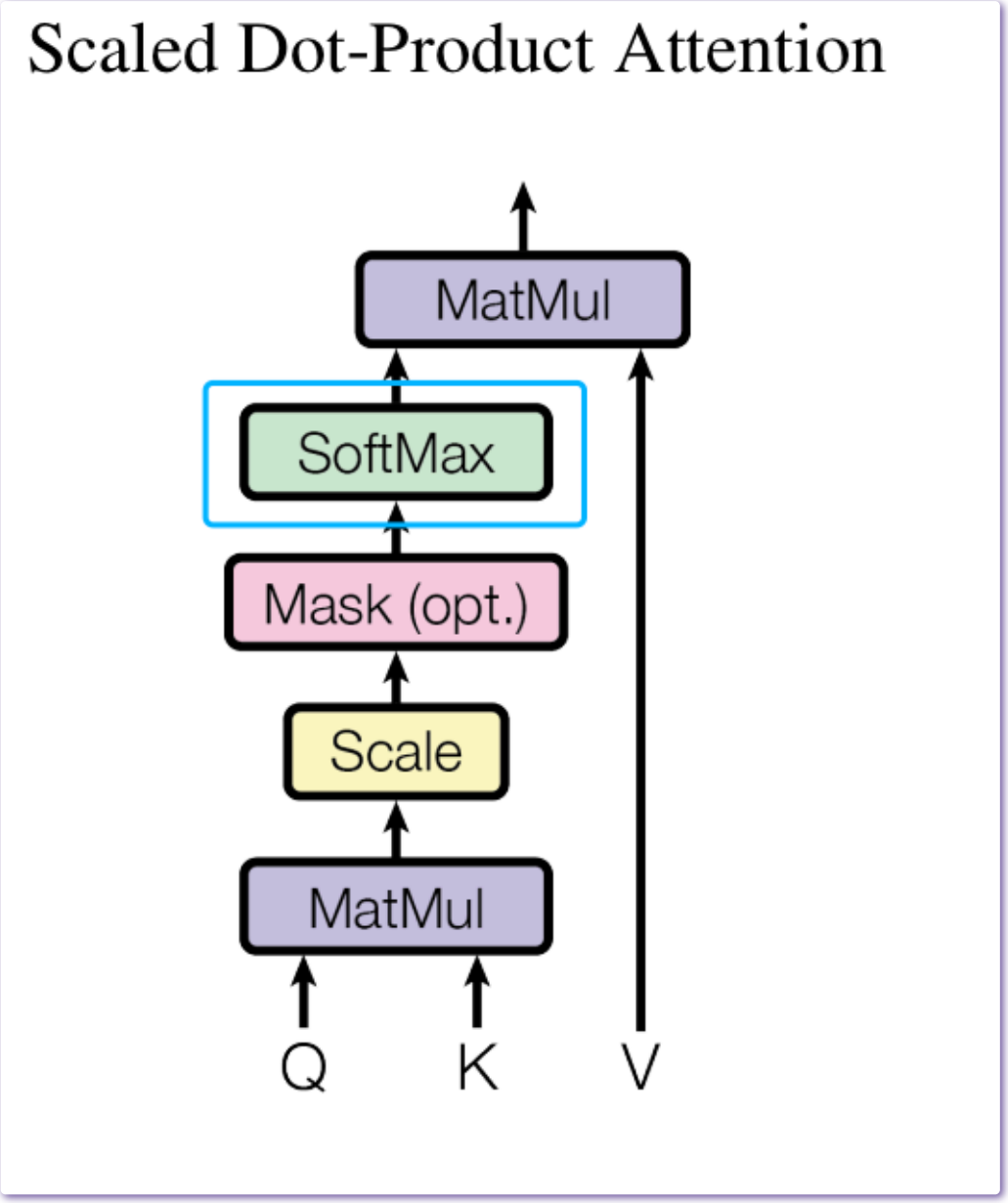
- 示例代码

```python
import torch
import torch.nn.functional as F

# 参数定义
batch_size = 4
num_heads = 8
seq_len_q = 10 # 目标序列长度
seq_len_kv = 12 # 源序列长度
embed_dim = 64

# 创建 Q, K, V 张量 (N, H, L/S, E/H)
# N: batch_size, H: num_heads, L: seq_len_q, S: seq_len_kv, E/H: head_dim
query = torch.randn(batch_size, num_heads, seq_len_q, embed_dim // num_heads)
key = torch.randn(batch_size, num_heads, seq_len_kv, embed_dim // num_heads)
value = torch.randn(batch_size, num_heads, seq_len_kv, embed_dim // num_heads)

# 场景1：标准注意力计算
output = F.scaled_dot_product_attention(query, key, value)
print("Standard Attention Output Shape:", output.shape)
"""
Standard Attention Output Shape: torch.Size([4, 8, 10, 8])
"""
```

```python

# 场景2：使用因果掩码 (Causal Mask) 进行自回归解码
# 通常在这种情况下，Q, K, V 的序列长度相同
causal_query = torch.randn(batch_size, num_heads, seq_len_q, embed_dim // num_heads)
output_causal = F.scaled_dot_product_attention(causal_query, causal_query, causal_query, is_causal=True)
print("\nCausal Attention Output Shape:", output_causal.shape)
"""
Causal Attention Output Shape: torch.Size([4, 8, 10, 8])
"""


# 场景3：使用自定义的注意力掩码 (Padding Mask)
# 假设 key/value 序列中有填充 (padding)
# 创建一个布尔掩码，True 代表这些位置是 padding，需要被忽略
attn_mask = torch.zeros(batch_size, num_heads, seq_len_q, seq_len_kv, dtype=torch.bool)
attn_mask[:, :, :, -2:] = True # 假设每个序列的最后两个 token 是 padding

output_masked = F.scaled_dot_product_attention(query, key, value, attn_mask=attn_mask)
print("\nMasked Attention Output Shape:", output_masked.shape)
"""
Masked Attention Output Shape: torch.Size([4, 8, 10, 8])
"""

```

15 十二、Softmax VS Safe-Softmax

- 在self-attention计算中，在Q和K矩阵计算完成后，还需要计算softmax，如下图



  但是直接计算softmax是有问题的，实际应用中我们需要计算的是safe-softmax
- 简单来说，**Safe-Softmax 和 Softmax 在数学定义上是完全等价的，区别仅在于代码实现层面，前者解决了后者在计算中可能遇到的数值溢出（Overflow）问题。**
- 核心区别: 数值稳定性 (Numerical Stability)

15.1 标准Softmax (理论公式)

- 我们先回顾一下Softmax的理论公式。对于一个向量 z=(z1,z2,...,zK)，其计算方式为：

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{26}$$

- **问题所在**：当输入的logits值（即 z_i）很大时，计算 e^{z_i} 可能会导致数值溢出。在计算机中，浮点数能表示的大小是有限的。例如，如果某个 z_i 是1000，那么 e^{1000} 将会是一个巨大无比的数字，远远超出

标准64位浮点数的表示范围，计算结果会变成 `inf` (无穷大)。当分子分母都变成 `inf` 时，最终结果就会是 `NaN` (Not a Number)，导致计算失败。

- **示例（天真的实现会导致失败）：**
    1. 输入Logits: `[1000, 1010, 990]` å
    2. 计算分子： `[e^1000, e^1010, e^990]` -> `[inf, inf, inf]`
    3. 计算分母： `inf + inf + inf` -> `inf`
    4. 最终结果： `[inf/inf, inf/inf, inf/inf]` -> `[NaN, NaN, NaN]`

15.2 Safe-Softmax (数值稳定的实现)

- 为了解决这个问题，Safe-Softmax在实现上增加了一个非常巧妙的技巧：在计算指数之前，从输入向量 z 的所有元素中减去该向量中的最大值。
- **实现公式:**
    1. **计算最大值**：$m = \max(z)$
    2. 使用下面这个等价的公式进行计算：

$$\text{Safe-Softmax}(z_i) = \frac{e^{z_i - m}}{\sum_{j=1}^{K} e^{z_j - m}} \tag{27}$$

- 为什么这是等价的?

  因为根据指数运算法则 $\frac{e^a}{e^b} = e^{a-b}$，我们可以把分子分母同时乘以一个常数$e^{-m}$：

$$\begin{aligned} \mathbf{Softmax(z_i)} &= \frac{e^{z_i}}{\sum e^{z_i}} \cdot \frac{e^{-m}}{e^{-m}} \\ &= \frac{e^{z_i} e^{-m}}{(\sum e^{z_i}) e^{-m}} \\ &= \frac{e^{z_i} e^{-m}}{\sum e^{z_i - m}} \end{aligned} \tag{28}$$

  数学上，结果完全不变。
- 为什么这样就安全了?

  通过减去最大值 m，新的向量 (z−m) 中的最大元素值变成了 m−m=0。所有其他元素都将是负数。
    1. $e^0 = 1$，这是一个非常安全的数字。
    2. 所有负数元素的指数 $e^{negative_value}$ 的结果将介于0和1之间。
    3. 这样就完美地避免了计算任何巨大数值的指数，从而防止了上溢（Overflow）问题。
- **示例（安全的实现可以成功）：**
    1. 输入Logits: `[1000, 1010, 990]`
    2. 找到最大值 m=1010
    3. 从每个元素中减去最大值: `[1000-1010, 1010-1010, 990-1010]` -> `[-10, 0, -20]`
    4. 计算指数: `[e^-10, e^0, e^-20]` -> `[4.5e-5, 1, 2.0e-9]` (都是很小的、安全的数字)
    5. 求和并相除，得到最终的、正确的概率分布。

15.3 总结对比

- **结论**

  你可以认为 **Safe-Softmax 就是 Softmax 在现实世界中的正确打开方式**。当你在TensorFlow、PyTorch等主流深度学习框架中使用 `nn.Softmax()` 或 `tf.nn.softmax()` 时，你使用的其实已经是内部实现好的Safe-Softmax了。

特性	标准 Softmax (理论/原生实现)	Safe-Softmax (实际/稳定实现)
数学形式	$\frac{e^{z_i}}{\sum e^{z_j}}$	$\frac{e^{z_i - \max(z)}}{\sum e^{z_j - \max(z)}}$
核心操作	直接计算指数	**先减最大值 \max(z) 再计算指数**
数值稳定性	❌ 易上溢 (Overflow)	✅ 无溢出风险
使用场景	理论推导	**所有实际框架实现** (PyTorch/TensorFlow)